STAR RX72N - rx_core Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Test List

Global rx_pin_from_pin (rx_port_pin_t pin)

test_rx_pin_from_pin() Unit tests verify all 144 port/pin combinations

Global rx_port_from_pin (rx_port_pin_t pin)

test_rx_port_from_pin() Unit tests verify all 144 port/pin combinations

Global rx_register_guard_get_correction_count (void)

Tested in test_rx_register_guard.c::test_correction_count_increment()

Tested in test_rx_register_guard.c::test_correction_count_before_init()

Global rx_register_guard_init (void)

Tested in test_rx_register_guard.c::test_init_captures_values()

Global rx_register_guard_is_initialized (void)

Tested in test_rx_register_guard.c::test_is_initialized_flag()

Tested in test_rx_register_guard.c::test_refresh_before_init()

Global rx_register_guard_refresh (void)

Tested in test_rx_register_guard.c::test_refresh_corrects_corruption()

Tested in test_rx_register_guard.c::test_refresh_performance()

Global rx_register_guard_reset_count (void)

Tested in test_rx_register_guard.c::test_reset_count()

Tested in test_rx_register_guard.c::test_reset_before_init()

Global rx_time_threadx_get_interface (rx_time_interface_t *iface)

Tested in test_rx_time_interface.c::test_validate_null_interface()

Tested in test_rx_time_interface.c::test_validate_missing_functions()

Tested in test_rx_time_interface.c::test_validate_valid_interface()

Tested in test_rx_time.c with nullptr and success cases

Chapter 2

Directory Hierarchy

2.1 Directories

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

Chapter 4

File Index

4.1 File List

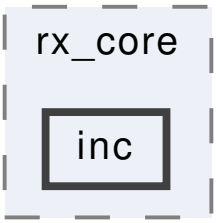
Here is a list of all files with brief descriptions:

Chapter 5

Directory Documentation

5.1 libs/rx_core/inc Directory Reference

Directory dependency graph for inc:



Files

- file rx_bit_constants.h

  Fundamental Bit and Byte Size Constants for Protocol and Data Manipulation.
- file rx_check.h

  Validation and Error Checking Macros for RX72N Firmware.
- file rx_err.h

  Error Code Definitions for RX72N Firmware.
- file rx_error_handler.h

  Error Handler Concrete Implementation with Retry Logic and Exponential Backoff.
- file rx_error_interface.h

  Abstract Error Handler Interface (Dependency Inversion Principle - SOLID).
- file rx_exception.h

  RX72N CPU Exception Handling.
- file rx_gpio_constants.h

  GPIO and System Constants for RX72N Microcontroller.
- file rx_infrastructure.h

  Global Infrastructure Initialization and Service Locator.
- file rx_iwdt.h

  Independent Watchdog Timer (IWDT) Driver for RX72N Microcontroller.

- file rx_log.h

  Type-Safe Logging System for RX72N Firmware with Zero-Overhead Compile-Time Filtering.
- file rx_pin_interface.h

  Abstract Pin Validator Interface - THE "I" and "D" in SOLID.
- file rx_pin_validator.h

  Concrete Pin Validator Implementation - Dependency Inversion Pattern.
- file rx_port_constants.h

  Centralized RX72N Port Number Definitions.
- file rx_register_guard.h

  Register Guard - Periodic Refresh for ESD/EMI Protection.
- file rx_register_protection.h

  RX72N Register Protection Control (PRCR) - Write Protection Constants.
- file rx_simulator_config.h

  Configuration for e^2 studio simulator support.
- file rx_threadx_config.h

  ThreadX RTOS Tick Rate Configuration.
- file rx_time_constants.h

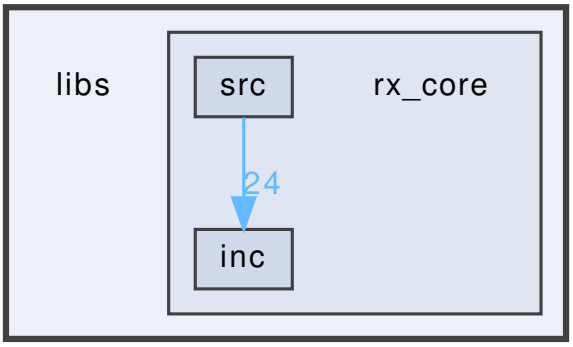
  Time Unit Conversion Constants for RX72N Firmware.
- file rx_time_interface.h

  Time Interface - Dependency Inversion for Timing Operations.
- file rx_wdt.h

  Watchdog Timer (WDT) Driver for RX72N Microcontroller.

5.2 libs Directory Reference

Directory dependency graph for libs:

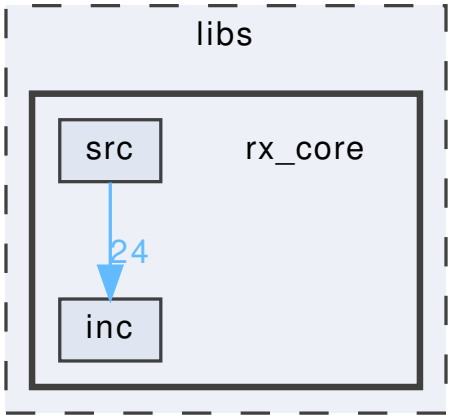


Directories

- directory rx_core

5.3 libs/rx_core Directory Reference

Directory dependency graph for rx_core:

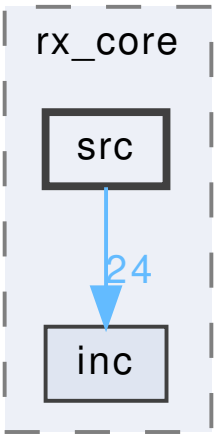


Directories

- directory inc
- directory src

5.4 libs/rx_core/src Directory Reference

Directory dependency graph for src:



Files

- file rx_error_handler.c

  Error Handler Concrete Implementation.
- file rx_exception.c

  RX72N CPU Exception Handling Implementation.
- file rx_infrastructure.c

  Global Infrastructure Initialization Implementation.
- file rx_iwdt.c

  Independent Watchdog Timer (IWDT) Implementation for RX72N.
- file rx_log_usb.c

  USB CDC logging backend implementation with boot buffering and thread safety.
- file rx_pin_validator.c

  Pin Validator Concrete Implementation - GPIO Pin Tracking and Conflict Prevention.
- file rx_register_guard.c

  Register Guard Implementation - ESD/EMI Protection for Critical Registers.
- file rx_time_threadx.c

  ThreadX Time Implementation for RX72N.
- file rx_wdt.c

  Watchdog Timer (WDT) Implementation for RX72N Microcontroller.

Chapter 6

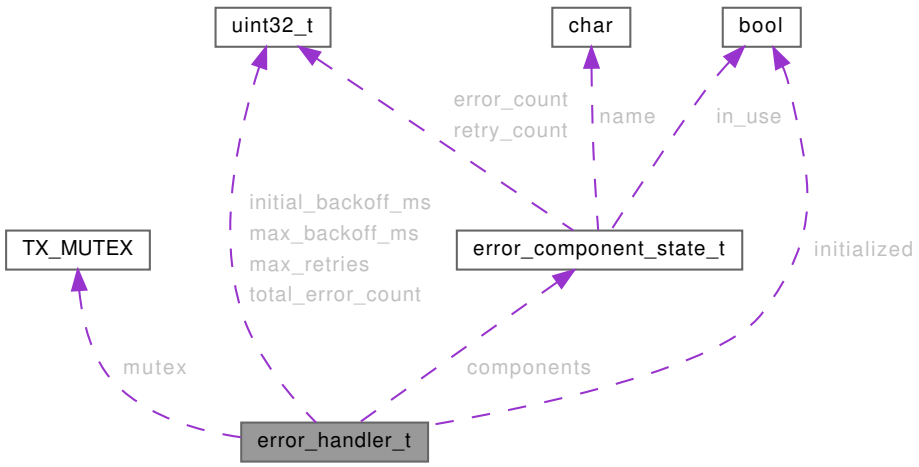
Data Structure Documentation

6.1 error_handler_t Struct Reference

Error Handler Concrete Implementation Structure (Main State Container).

#include <rx_error_handler.h>

Collaboration diagram for error_handler_t:



Data Fields

- TX_MUTEX mutex
- uint32_t total_error_count
- error_component_state_t components [k_error_handler_max_components]
- uint32_t max_retries
- uint32_t initial_backoff_ms
- uint32_t max_backoff_ms
- bool initialized

6.1.1 Detailed Description

Error Handler Concrete Implementation Structure (Main State Container).

Complete error handler state including ThreadX mutex, component tracking array, retry configuration, and initialization flag. This structure implements the rx_error_interface_t abstraction, providing concrete error tracking and retry logic.

6.1.1.1 Purpose and Responsibility

This structure serves as the **central error management database** for the entire system:

1. **Thread Safety**: TX_MUTEX protects concurrent access
2. **Global Statistics**: total_error_count aggregates all component errors
3. **Component Tracking**: components[] array isolates per-component state
4. **Retry Strategy**: max_retries, backoff parameters define retry behavior
5. **Lifecycle Management**: initialized flag prevents double-init

6.1.1.2 Memory Layout

Offset	Size	Field	Type	Alignment	Notes
0	∼100	mutex	TX_MUTEX	4	ThreadX mutex (size varies by config)
100	4	total_error_count	uint32_t	4	Global error counter
104	704	components[16]	array	4	16 x 44 bytes per component
808	4	max_retries	uint32_t	4	Retry limit configuration
812	4	initial_backoff_ms	uint32_t	4	Initial backoff delay
816	4	max_backoff_ms	uint32_t	4	Maximum backoff cap
820	1	initialized	bool	1	Initialization flag
821	3	(padding)	-	-	Compiler alignment
Total	**∼924 bytes**				Structure size (varies by TX_MUTEX)

Note: Actual size depends on ThreadX TX_MUTEX size, which varies by configuration. Typical total: ∼1.0 to 1.3 KB.

6.1.1.3 Initialization Requirements

Before use, this structure MUST be initialized via error_handler_init():

1. TX_MUTEX must be created (tx_mutex_create)
2. total_error_count set to 0
3. All components[] marked in_use=false
4. Retry parameters copied from config
5. initialized flag set to true

6.1.1.4 Typical Lifecycle

[UNINITIALIZED] (all fields undefined)
v error_handler_init(&handler, &config)
[INITIALIZED] (mutex created, counters zeroed, initialized=true)
v normal operation (report_error, retry checks)
[IN_USE] (components registered, counters incrementing)
v error_handler_deinit(&handler)
[DEINITIALIZED] (mutex deleted, initialized=false)

Field Constraints:

Field	Valid Values	Invariant	Notes
mutex	Valid TX_MUTEX	Created if initialized=true	ThreadX owned
total_error_count	0 to 2^32-1	Sum of all components[].error_count	Wraps on overflow
components[]	See error_component_state_t	in_use=true slots <= 16	Static array
max_retries	0 to 2^32-1	0 = unlimited retries	From config
initial_backoff_ms	1 to max_backoff_ms	> 0 for meaningful backoff	From config
max_backoff_ms	>= initial_backoff_ms	Caps exponential growth	From config
initialized	true/false	true after init, false after deinit	Lifecycle flag

Usage Example (Declaration and Initialization):

```
// Static allocation (typical for embedded systems)
static error_handler_t s_global_error_handler;

void system_init(void) {
  // Configure error handler
  const error_handler_config_t config = {
    .max_retries        = 3,      // Up to 3 retries
    .initial_backoff_ms = 10,     // Start at 10ms
    .max_backoff_ms     = 5000    // Cap at 5 seconds
  };

  // Initialize
  rx_err_t err = error_handler_init(&s_global_error_handler, &config);
  if (err != k_rx_ok) {
    uart_debug_puts("[FATAL] Error handler init failed\r\n");
    while (1);  // Cannot proceed without error handling
  }

  // Get interface for use by high-level modules
  rx_error_interface_t error_iface;
  error_handler_get_interface(&error_iface, &s_global_error_handler);

  // Inject interface into other modules
  motor_controller_init(&error_iface);
  spi_driver_init(&error_iface);
}
```

Usage Example (Direct Access - Not Recommended):

```
// Direct field access (bypasses interface - violates DIP)
// USE INTERFACE INSTEAD!

// Get total error count (for diagnostics only)
uint32_t total_errors = s_global_error_handler.total_error_count;
uart_debug_printf("Total system errors: %lu\r\n", total_errors);

// Iterate components for diagnostics
for (uint8_t i = 0; i < k_error_handler_max_components; i++) {
  if (s_global_error_handler.components[i].in_use) {
    uart_debug_printf("[%s] errors:%lu retries:%lu\r\n",
```

```
        s_global_error_handler.components[i].name,
        s_global_error_handler.components[i].error_count,
        s_global_error_handler.components[i].retry_count);
    }
  }
```

Invariant

initialized=true -> mutex is valid AND total_error_count defined

total_error_count >= sum of all components[].error_count (approximately)

max_backoff_ms >= initial_backoff_ms (always)

Number of in_use=true components <= k_error_handler_max_components

Note

Structure must be statically allocated or have lifetime exceeding all users

Typical allocation: File-scope static (static error_handler_t s_handler)

Memory footprint: ~1.0-1.3 KB depending on ThreadX configuration

Warning

Never copy this structure (contains TX_MUTEX which is not copyable)

Direct field access violates Dependency Inversion - use interface

Must call error_handler_deinit() before destruction (mutex cleanup)

Attention

TX_MUTEX size varies by ThreadX configuration (affects total size)

See also

error_handler_init() Initializes this structure

error_handler_get_interface() Converts to abstract interface

error_handler_deinit() Cleans up resources (mutex deletion)

rx_error_interface_t Abstract interface (use this, not direct access)

Since

Version 1.0.0

Definition at line 621 of file rx_error_handler.h.

6.1.2 Field Documentation

6.1.2.1 components

error_component_state_t error_handler_t::components[k_error_handler_max_components]

Per-component error tracking array. Maximum 16 components (k_error_handler_max_components). Each slot tracks one component's errors and retry state. Linear search for component name on each access

Definition at line 626 of file rx_error_handler.h.

Referenced by impl_clear_errors(), internal_find_component(), and internal_find_or_create_component().

6.1.2.2 initial_backoff_ms

uint32_t error_handler_t::initial_backoff_ms

Initial backoff delay in milliseconds for first retry. Base value for exponential backoff calculation: delay = initial x 2^retry_count. Typical values: 10ms to 100ms. Must be > 0 for meaningful backoff

Definition at line 631 of file rx_error_handler.h.

Referenced by error_handler_init(), and impl_get_backoff_delay().

6.1.2.3 initialized

bool error_handler_t::initialized

Initialization flag. true = error_handler_init() succeeded, structure is ready for use. false = Not initialized or deinitialized (error_handler_deinit() called). Check before all operations

Definition at line 635 of file rx_error_handler.h.

Referenced by error_handler_deinit(), error_handler_get_interface(), error_handler_init(), impl_clear_errors(), impl_get_backoff_delay(), impl_get_component_error_count(), impl_get_error_count(), impl_is_retry_limit_reached(), impl_report_error(), and impl_reset_retry_counter().

6.1.2.4 max_backoff_ms

uint32_t error_handler_t::max_backoff_ms

Maximum backoff delay cap in milliseconds. Prevents exponential backoff from growing unbounded. Typical values: 5000ms to 60000ms. Must be >= initial_backoff_ms. Limits delay = min(initial x 2^retry, max)

Definition at line 633 of file rx_error_handler.h.

Referenced by error_handler_init(), and impl_get_backoff_delay().

6.1.2.5 max_retries

uint32_t error_handler_t::max_retries

Maximum retry attempts before giving up. 0 = unlimited retries (use with caution). Copied from error_handler_config_t during init. Compared against components[].retry_count to determine failure

Definition at line 629 of file rx_error_handler.h.

Referenced by error_handler_init(), impl_get_backoff_delay(), and impl_is_retry_limit_reached().

6.1.2.6 mutex

TX_MUTEX error_handler_t::mutex

ThreadX mutex for thread-safe access to all fields. Created by error_handler_init(), deleted by error_handler_deinit(). Protects concurrent report_error() and retry check calls from multiple tasks

Definition at line 623 of file rx_error_handler.h.

Referenced by error_handler_deinit(), error_handler_init(), impl_clear_errors(), impl_get_backoff_delay(), impl_get_component_error_count(), impl_get_error_count(), impl_is_retry_limit_reached(), impl_report_error(), and impl_reset_retry_counter().

6.1.2.7 total_error_count

uint32_t error_handler_t::total_error_count

Aggregate error count across ALL components. Incremented on each report_error() call regardless of component. Used for system-wide health monitoring and diagnostics. Range: 0 to 2^32-1 (wraps on overflow)

Definition at line 625 of file rx_error_handler.h.

Referenced by impl_clear_errors(), impl_get_error_count(), and impl_report_error().

The documentation for this struct was generated from the following file:

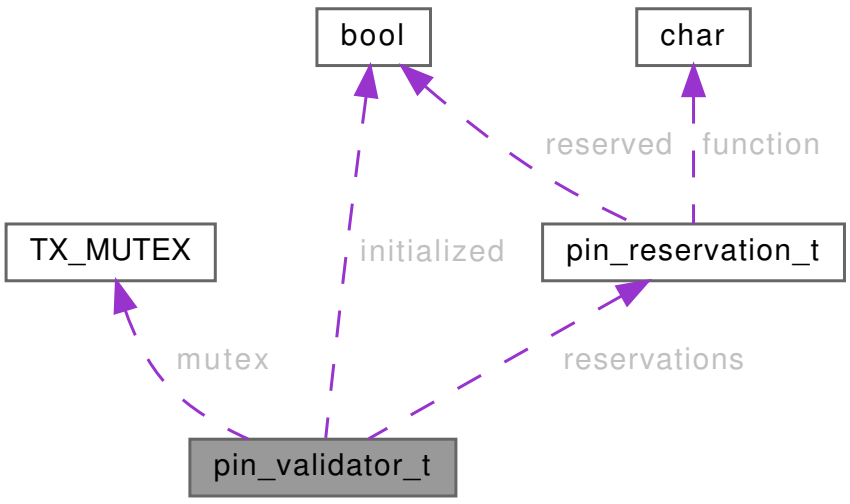
- libs/rx_core/inc/rx_error_handler.h

6.2 pin_validator_t Struct Reference

Concrete Pin Validator Implementation Structure.

#include <rx_pin_validator.h>

Collaboration diagram for pin_validator_t:



Data Fields

- TX_MUTEX mutex

  ThreadX mutex for thread-safe access to reservation table.
- pin_reservation_t reservations [k_pin_validator_max_ports][k_pin_validator_max_pins]

  2D array of pin reservation states [port][pin]
- bool initialized

  Lifecycle flag indicating validator is ready for use.

6.2.1 Detailed Description

Concrete Pin Validator Implementation Structure.

This is the **concrete implementation** of the pin validator that backs the rx_pin_interface_t abstract interface. Contains all state for pin validation, reservation tracking, and thread synchronization.

6.2.1.1 Memory Layout

```
struct pin_validator_t {
  // Offset | Size  | Field
  //      0 |  ~100 | TX_MUTEX mutex
  //   ~100 |  4488 | pin_reservation_t[17][8] reservations
  //  ~4588 |     1 | bool initialized
  //  ~4589 |    ~3 | (padding to alignment)
  // TOTAL: ~4592 bytes (~4.5 KB)
};
```

6.2.1.2 Field Descriptions

6.2.1.2.1 mutex (ThreadX Mutex)

- **Purpose**: Protects reservation table from concurrent access
- **Type**: TX_MUTEX (ThreadX RTOS primitive)
- **Size**: ∼100 bytes (platform-dependent)
- **Operations protected**:
  - reserve_pin() - Exclusive write access
  - release_pin() - Exclusive write access
  - is_pin_reserved() - Shared read access
  - get_pin_function() - Shared read access + string copy
  - clear_all_reservations() - Exclusive write access

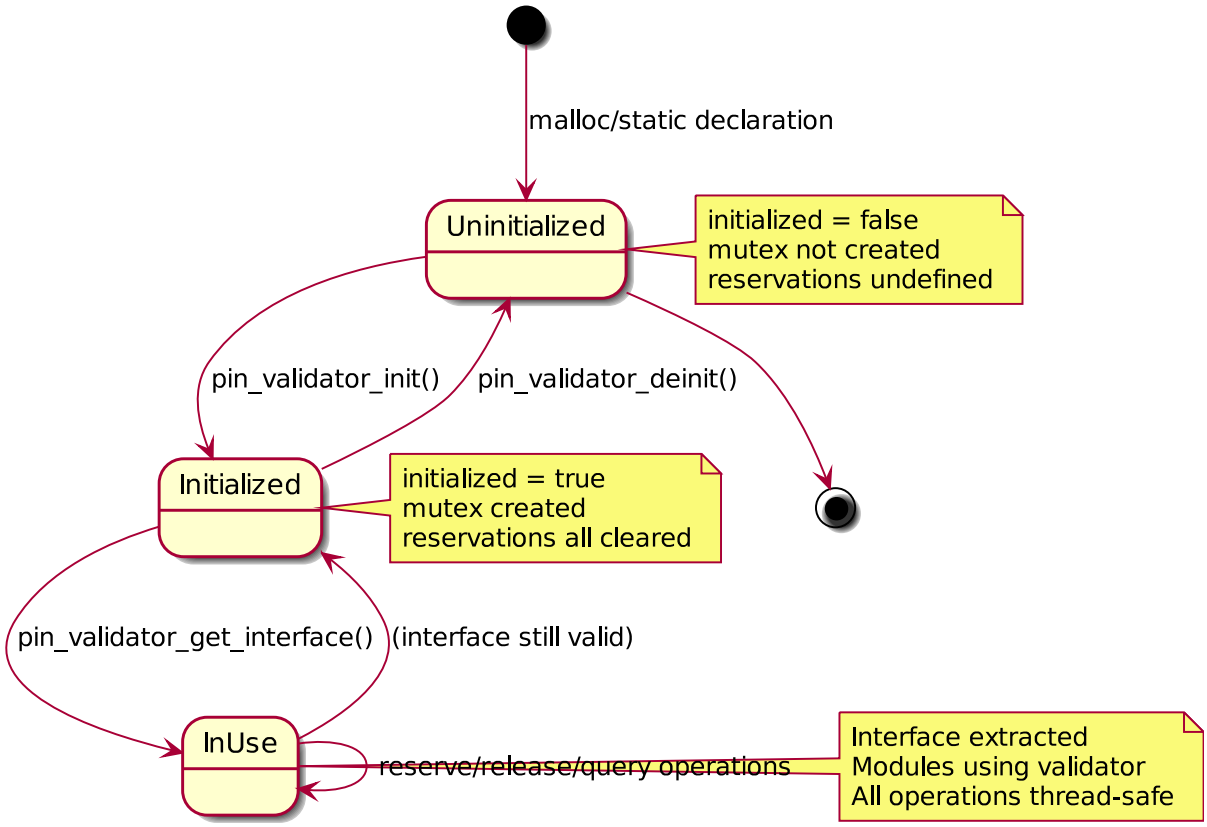
6.2.1.2.2 reservations[port][pin] (2D Array)

- **Purpose**: Stores reservation status for all pins
- **Dimensions**: 17 ports x 8 pins = 136 slots
- **Per-slot size**: 33 bytes (1 bool + 32 char)
- **Total size**: 136 x 33 = 4488 bytes (∼4.4 KB)
- **Indexing**: reservations[port][pin] where:
  - port: 0-9 (decimal), 10-16 (hex A-G)
  - pin: 0-7
- **Access pattern**: O(1) direct indexing, no search needed

6.2.1.2.3 initialized (Lifecycle Flag)

- **Purpose**: Tracks whether pin_validator_init() was called
- **Type**: bool (1 byte)
- **States**:
  - false: Validator not initialized, cannot use
  - true: Validator ready, mutex created, table zeroed
- **Use**: Prevents double-init and validates before interface extraction

6.2.1.3 Lifecycle State Machine



6.2.1.4 Typical Usage Pattern

```
// Step 1: Declare (uninitialized state)
static pin_validator_t s_validator;  // .bss section, zeroed by startup code

// Step 2: Initialize (transition to initialized state)
rx_err_t err = pin_validator_init(&s_validator);
if (err != k_rx_ok) {
  return err;  // Initialization failed
}
// Now: s_validator.initialized == true
//      s_validator.mutex created
//      s_validator.reservations[] all cleared

// Step 3: Extract interface (transition to in-use state)
rx_pin_interface_t iface;
err = pin_validator_get_interface(&iface, &s_validator);
if (err != k_rx_ok) {
  return err;
}
```

```
// Now: iface.ctx points to s_validator
//      iface function pointers point to validator's implementations

// Step 4: Use through interface (in-use state)
err = iface.reserve_pin(iface.ctx, 0xA, 5, "SPI_COPI");
// Internally: Acquires mutex, checks reservations[10][5], updates if free

// Step 5: Deinitialize (transition back to uninitialized)
pin_validator_deinit(&s_validator);
// Now: s_validator.initialized == false
//      mutex destroyed
//      reservations cleared (optional)
```

6.2.1.5 Memory Allocation Strategy

This struct is **ALWAYS statically allocated** (never malloc):

- Global/file scope: static pin_validator_t s_validator;
- Infrastructure module: static pin_validator_t s_global_validator;
- Stack allocation: pin_validator_t validator; (rare, wastes stack)

Why static allocation?

- NASA Rule 3: No dynamic memory after initialization
- Predictable memory layout
- No fragmentation risk
- No malloc/free errors

6.2.1.6 Thread Safety Responsibilities

Component	Responsibility
Caller	Ensure init/deinit called from single thread
Validator	Protect all reservation table access with mutex
Interface methods	Acquire mutex before accessing reservations[]
ThreadX	Handle mutex priority inheritance

Definition at line 596 of file rx_pin_validator.h.

6.2.2 Field Documentation

6.2.2.1 initialized

bool pin_validator_t::initialized

Lifecycle flag indicating validator is ready for use.

Is the validator initialized?

Set to true by pin_validator_init(), false by pin_validator_deinit(). Checked by pin_validator_get_interface() to ensure validator is ready before extracting interface.

Note

Total size: ~4592 bytes (~4.5 KB)

Always use static allocation (global or file scope)

Mutex must be created before use (via pin_validator_init())

Thread-safe after initialization (all operations mutex-protected)

See also

pin_validator_init() Initialize this structure

pin_validator_get_interface() Extract interface from this structure

pin_validator_deinit() Clean up this structure

rx_pin_interface_t Abstract interface this implements

Since

Version 1.0.0

Definition at line 601 of file rx_pin_validator.h.

Referenced by pin_validator_deinit(), pin_validator_get_interface(), and pin_validator_init().

6.2.2.2 mutex

TX_MUTEX pin_validator_t::mutex

ThreadX mutex for thread-safe access to reservation table.

ThreadX mutex for thread-safe operation

Created during pin_validator_init(), destroyed during pin_validator_deinit(). Protects all accesses to the reservations array. Uses priority inheritance to prevent priority inversion.

Definition at line 597 of file rx_pin_validator.h.

Referenced by impl_clear_all_reservations(), impl_get_pin_function(), impl_is_pin_reserved(), impl_release_pin(), impl_reserve_pin(), pin_validator_deinit(), and pin_validator_init().

6.2.2.3 reservations

pin_reservation_t pin_validator_t::reservations[k_pin_validator_max_ports][k_pin_validator_max_pins]

2D array of pin reservation states [port][pin]

Pin reservation tracking array (indexed: reservations[port][pin], Port 0-9: Decimal, Port 10-16: Hex A-G)

Direct-mapped table: port number is first index, pin number is second index. Example: $\text{reservations}[0\text{xA}][5]$ = Port A, Pin 5. All access must be mutex-protected.

Definition at line 598 of file rx_pin_validator.h.

Referenced by impl_clear_all_reservations(), impl_get_pin_function(), impl_is_pin_reserved(), impl_release_pin(), and impl_reserve_pin().

The documentation for this struct was generated from the following file:

- libs/rx_core/inc/rx_pin_validator.h

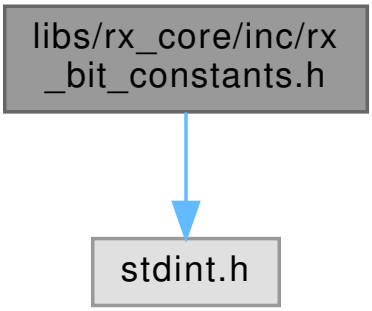
Chapter 7

File Documentation

7.1 libs/rx_core/inc/rx_bit_constants.h File Reference

Fundamental Bit and Byte Size Constants for Protocol and Data Manipulation.

#include <stdint.h>
Include dependency graph for rx_bit_constants.h:



Enumerations

- enum rx_bit_sizes_t : uint8_t { k_rx_bits_per_byte , k_rx_bits_per_word16 , k_rx_bits_per_word32 , k_rx_bits_per_word64 }

  Fundamental bit and byte size constants for data type sizes.
- enum rx_bit_shifts_t : uint8_t { k_rx_shift_byte , k_rx_shift_word16 , k_rx_shift_word24 }

  Bit shift constants for byte and word extraction from multi-byte values.

7.1.1 Detailed Description

Fundamental Bit and Byte Size Constants for Protocol and Data Manipulation.

7.1.1.1 Overview

Centralized repository of typed enum constants for bit/byte sizes and shift operations used throughout the STAR firmware. Provides self-documenting, type-safe constants for:

- Protocol encoding/decoding (Protocol Buffers, ASCII frames, CRC)
- Data serialization and deserialization
- Bit manipulation (extraction, packing, masking)
- Endianness conversion (big-endian <-> little-endian)
- Buffer size calculations

Design Philosophy:

- **No magic numbers**: All numeric literals replaced with named typed enums (NASA Rule 8)
- **Type safety**: C23 typed enums with explicit underlying types (uint8_t)
- **Self-documenting**: Constants have descriptive names (k_rx_bits_per_byte vs. literal 8)
- **Searchable**: grep for "k_rx_shift_byte" finds all byte-shift operations
- **Hardware-agnostic**: These are universal constants, not hardware-specific

7.1.1.2 Scope and Usage

INCLUDE in:

- Protocol encoding/decoding modules (rx_frame, rx_nanopb, rx_crc)
- Communication layers (rx_spi_comm, rx_usb_comm, rx_comm_manager)
- Utility functions for data packing/unpacking
- Unit conversion functions

EXCLUDE from (use peripheral-specific headers instead):

- Hardware register bit positions -> Use rx72n_*_regs.h (e.g., USBE.SYSCFG.DRPD = bit 5)
- Clock divider calculations -> Use rx72n_clock.h (e.g., PCLKA_DIV = 2)
- Peripheral-specific shifts -> Define in peripheral header (e.g., ADC channel shift)

7.1.1.3 Common Usage Patterns

7.1.1.3.1 1. Bit-to-Byte Conversion (Ceiling Division)

```
// Calculate bytes needed to store N bits (round up)
uint32_t bit_count = 100;  // 100 bits
uint32_t byte_count = (bit_count + k_rx_bits_per_byte - 1) / k_rx_bits_per_byte;
// Result: (100 + 8 - 1) / 8 = 107 / 8 = 13 bytes (ceiling division)
```

7.1.1.3.2 2. Byte Extraction from Multi-Byte Word

```
// Extract individual bytes from 32-bit word (little-endian)
uint32_t word = 0x12345678;
uint8_t byte0 = (word » (k_rx_shift_byte * 0)) & 0xFF;  // 0x78 (LSB)
uint8_t byte1 = (word » (k_rx_shift_byte * 1)) & 0xFF;  // 0x56
uint8_t byte2 = (word » (k_rx_shift_byte * 2)) & 0xFF;  // 0x34
uint8_t byte3 = (word » k_rx_shift_word24) & 0xFF;      // 0x12 (MSB)
```

7.1.1.3.3 3. Big-Endian 16-bit Packing

```
// Pack uint16_t into big-endian byte array
uint16_t value = 0xABCD;
uint8_t buffer[2];
buffer[0] = (value » k_rx_shift_byte) & 0xFF;  // 0xAB (high byte)
buffer[1] = (value » 0) & 0xFF;                // 0xCD (low byte)
```

7.1.1.3.4 4. Buffer Size Calculation for Bit Fields

```
// Allocate buffer for protocol with bit-stuffing
const uint32_t k_payload_bits = 512;  // 512 bits of data
const uint32_t k_overhead_bits = 32;  // CRC-32
const uint32_t k_total_bits = k_payload_bits + k_overhead_bits;

// Calculate bytes needed (ceiling division)
const uint32_t k_buffer_size =
    (k_total_bits + k_rx_bits_per_byte - 1) / k_rx_bits_per_byte;
uint8_t buffer[k_buffer_size];  // 68 bytes
```

7.1.1.4 Hardware Requirements

Requirement	Value	Notes
CPU	Any (RX, ARM, x86)	Universal constants, platform-agnostic
Compiler	C23 or later	Requires typed enum support : uint8_t
Dependencies	stdint.h	For uint8_t underlying type
RAM	0 bytes	Compile-time constants (no storage)
Flash	0 bytes	Inlined by compiler, no code generated

NASA Power of 10 Compliance:

- **Rule 1 [YES]:** No goto, setjmp/longjmp, recursion (header-only, no control flow)
- **Rule 2 [N/A]:** No loops (header-only)
- **Rule 3 [YES]:** No dynamic allocation (compile-time constants)
- **Rule 4 [N/A]:** No functions (header-only)
- **Rule 5 [N/A]:** No functions (header-only)
- **Rule 6 [YES]:** Data at smallest scope (file-scoped enums)
- **Rule 7 [N/A]:** No functions (header-only)
- **Rule 8 [YES]:** No macros except include guards (uses typed enums instead)
- **Rule 9 [YES]:** No pointers (value types only)
- **Rule 10 [YES]:** Compiles with -Wall -Wextra -Werror

SOLID Principles:

- **S (Single Responsibility):** Module provides ONLY universal bit/byte constants
- **O (Open/Closed):** Extensible (add new enums) without modifying existing code
- **L (Liskov Substitution):** N/A (no functions, only data)
- **I (Interface Segregation):** Minimal API (2 enums), each with focused purpose
- **D (Dependency Inversion):** No dependencies (standalone header)

Module Dependencies:

```
rx_bit_constants.h
  +--> stdint.h (uint8_t for underlying enum type)

Used by:
  +--> lib/rx_frame/ (protocol framing, byte packing)
  +--> lib/rx_crc/ (CRC-32 byte-wise operations)
  +--> lib/rx_nanopb/ (Protocol Buffer serialization)
  +--> lib/rx_comm_manager/ (communication buffer management)
  +--> lib/rx_hal/src/uart.c (data serialization)
```

Author

STAR Team

Date

2026-01-27

Copyright

MIT License

See also

stdint.h for standard integer types

rx_gpio_constants.h for GPIO pin constants

rx_port_constants.h for port/pin mapping constants

rx_time_constants.h for time conversion constants

Version

1.0.0

Since

Version 1.0.0

Definition in file rx_bit_constants.h.

7.1.2 Enumeration Type Documentation

7.1.2.1 rx_bit_shifts_t

enum rx_bit_shifts_t : uint8_t

Bit shift constants for byte and word extraction from multi-byte values.

Provides named constants for common bit-shift operations used in protocol encoding, endianness conversion, and byte-wise data access. These constants make code self- documenting and eliminate magic shift values.

Underlying Type: uint8_t (8-bit unsigned) - sufficient for all values (max = 24) **Relationship to rx_bit_sizes_t:**

- k_rx_shift_byte = k_rx_bits_per_byte (both = 8)
- k_rx_shift_word16 = k_rx_bits_per_word16 (both = 16)
- k_rx_shift_word24 = 3 x k_rx_bits_per_byte (3 bytes = 24 bits)

7.1.2.2 Usage Patterns

7.1.2.2.1 Byte Extraction from 32-bit Word (Little-Endian)

```
uint32_t word = 0x12345678;  // Little-endian representation

// Extract individual bytes
uint8_t byte0 = (word » 0) & 0xFF;                 // 0x78 (LSB, byte 0)
uint8_t byte1 = (word » k_rx_shift_byte) & 0xFF;   // 0x56 (byte 1)
uint8_t byte2 = (word » k_rx_shift_word16) & 0xFF; // 0x34 (byte 2)
uint8_t byte3 = (word » k_rx_shift_word24) & 0xFF; // 0x12 (MSB, byte 3)
```

7.1.2.2.2 Big-Endian 32-bit Packing

```
// Pack uint32_t into big-endian byte array
uint32_t value = 0xAABBCCDD;
uint8_t buffer[4];

buffer[0] = (value » k_rx_shift_word24) & 0xFF;  // 0xAA (MSB first)
buffer[1] = (value » k_rx_shift_word16) & 0xFF;  // 0xBB
buffer[2] = (value » k_rx_shift_byte) & 0xFF;    // 0xCC
buffer[3] = (value » 0) & 0xFF;                  // 0xDD (LSB last)
```

7.1.2.2.3 Big-Endian 16-bit Packing/Unpacking

```
// Pack uint16_t -> big-endian buffer
uint16_t value = 0xABCD;
uint8_t buffer[2];
buffer[0] = (value » k_rx_shift_byte) & 0xFF;  // 0xAB (high byte)
buffer[1] = (value » 0) & 0xFF;                // 0xCD (low byte)

// Unpack big-endian buffer -> uint16_t
uint16_t reconstructed = ((uint16_t)buffer[0] « k_rx_shift_byte) | buffer[1];
// Result: (0xAB « 8) | 0xCD = 0xABCD
```

7.1.2.3 Why Not Use rx_bit_sizes_t Directly?

Semantic Clarity:

- k_rx_shift_byte **clearly indicates** "shift to access next byte"
- k_rx_bits_per_byte **ambiguously suggests** "number of bits in a byte"
- Using k_rx_shift_byte makes the **intent explicit** (shift operation)

Example:

```
// CLEAR INTENT: Using k_rx_shift_byte
uint8_t byte1 = (word » k_rx_shift_byte) & 0xFF;  // [OK] Obvious: shift by 1 byte

// AMBIGUOUS INTENT: Using k_rx_bits_per_byte
uint8_t byte1 = (word » k_rx_bits_per_byte) & 0xFF;  // [X] Less clear: shifting? counting?
```

7.1.2.4 Value Justification Table

Constant	Value	Derivation	Use Case
k_rx_shift_byte	8	1 byte x 8 bits	Access byte N in multi-byte word

Constant	Value	Derivation	Use Case
k_rx_shift_word16	16	2 bytes x 8 bits	Access upper 16 bits of 32-bit word
k_rx_shift_word24	24	3 bytes x 8 bits	Access MSB (byte 3) of 32-bit word

Usage Example: CRC-32 Byte-Wise Processing

```
#include "rx_bit_constants.h"

// Extract bytes from CRC-32 for transmission (big-endian)
void crc32_to_bytes(uint32_t crc, uint8_t* buffer) {
  buffer[0] = (crc » k_rx_shift_word24) & 0xFF;  // Byte 3 (MSB)
  buffer[1] = (crc » k_rx_shift_word16) & 0xFF;  // Byte 2
  buffer[2] = (crc » k_rx_shift_byte) & 0xFF;    // Byte 1
  buffer[3] = (crc » 0) & 0xFF;                  // Byte 0 (LSB)
}

// Reconstruct CRC-32 from big-endian byte array
uint32_t bytes_to_crc32(const uint8_t* buffer) {
  return ((uint32_t)buffer[0] « k_rx_shift_word24) |  // MSB
         ((uint32_t)buffer[1] « k_rx_shift_word16) |
         ((uint32_t)buffer[2] « k_rx_shift_byte) |
         ((uint32_t)buffer[3] « 0);                  // LSB
}
```

Usage Example: Protocol Field Extraction

```
// Extract 16-bit field from Protocol Buffer varint
uint32_t varint = 0x12345678;
uint16_t field = (varint » k_rx_shift_word16) & 0xFFFF;
// Result: 0x1234 (upper 16 bits)
```

See also

rx_bit_sizes_t for related bit count constants

k_rx_bits_per_byte equivalent value to k_rx_shift_byte (both = 8)

k_rx_bits_per_word16 equivalent value to k_rx_shift_word16 (both = 16)

Since

Version 1.0.0

Enumerator

k_rx_shift_byte	Shift amount to access next byte (8 bits = 1 byte). Use for extracting byte N from multi-byte word. Example: byte1 = (word >> k_rx_shift_byte) & 0xFF. Equivalent to k_rx_bits_per_byte but semantically clearer for shift operations.
k_rx_shift_word16	Shift amount to access upper 16 bits of 32-bit word (16 bits = 2 bytes). Use for extracting high word from dword. Example: high_word = (dword >> k_rx_shift_word16) & 0xFFFF. Also used for big-endian 16-bit packing/unpacking.
k_rx_shift_word24	Shift amount to access byte 3 (MSB) of 32-bit word (24 bits = 3 bytes). Use for extracting most significant byte. Example: msb = (word >> k_rx_shift_word24) & 0xFF. Critical for big-endian 32-bit encoding.

Definition at line 352 of file rx_bit_constants.h.

```
00352             : uint8_t {
00353   k_rx_shift_byte =
```

```
00354     8, /**< Shift amount to access next byte (8 bits = 1 byte). Use for extracting byte N from multi-byte word. Example:
          byte1 = (word » k_rx_shift_byte) & 0xFF. Equivalent to k_rx_bits_per_byte but semantically clearer for shift
          operations. */
00355   k_rx_shift_word16 =
00356     16, /**< Shift amount to access upper 16 bits of 32-bit word (16 bits = 2 bytes). Use for extracting high word from
          dword. Example: high_word = (dword » k_rx_shift_word16) & 0xFFFF. Also used for big-endian 16-bit
          packing/unpacking. */
00357   k_rx_shift_word24 =
00358     24, /**< Shift amount to access byte 3 (MSB) of 32-bit word (24 bits = 3 bytes). Use for extracting most significant
          byte. Example: msb = (word » k_rx_shift_word24) & 0xFF. Critical for big-endian 32-bit encoding. */
00359 } rx_bit_shifts_t;
```

7.1.2.5 rx_bit_sizes_t

enum rx_bit_sizes_t : uint8_t

Fundamental bit and byte size constants for data type sizes.

Defines universal constants for bit counts in standard data types (byte, word16, word32, word64). These constants eliminate magic numbers in protocol encoding, buffer calculations, and bit manipulation operations.

Underlying Type: uint8_t (8-bit unsigned) - sufficient for all values (max = 64) **Guaranteed Sizes:** All values are compile-time constants (no runtime overhead)

7.1.2.6 Usage Patterns

7.1.2.6.1 Ceiling Division for Bit-to-Byte Conversion

```
// Calculate bytes needed for N bits (always round up)
uint32_t bit_count = 100;  // Example: 100 bits
uint32_t byte_count = (bit_count + k_rx_bits_per_byte - 1) / k_rx_bits_per_byte;
// Result: (100 + 7) / 8 = 13 bytes (ceiling of 100/8 = 12.5)
```

7.1.2.6.2 Byte Indexing in Multi-Byte Words

```
// Extract byte N from 32-bit word
uint32_t word = 0xDEADBEEF;
uint8_t byte_index = 2;  // Want byte 2 (0xBE)
uint8_t byte = (word » (k_rx_bits_per_byte * byte_index)) & 0xFF;
// Result: (0xDEADBEEF » 16) & 0xFF = 0xBE
```

7.1.2.6.3 Protocol Buffer Sizing

```
// Allocate buffer for protocol with CRC-32
const uint32_t k_payload_bytes = 64;
const uint32_t k_crc_bits = k_rx_bits_per_word32;  // CRC-32 = 32 bits
const uint32_t k_crc_bytes = k_crc_bits / k_rx_bits_per_byte;  // 4 bytes
const uint32_t k_total_bytes = k_payload_bytes + k_crc_bytes;  // 68 bytes
uint8_t buffer[k_total_bytes];
```

7.1.2.7 Value Justification Table

Constant	Value	Rationale	Standard Reference
k_rx_bits_per_byte	8	Universal (ANSI, ISO/IEC, POSIX)	C99 Sec.3.6: CHAR_BIT = 8
k_rx_bits_per_word16	16	uint16_t guaranteed size	C99 Sec.7.18.1.1 exact-width types
k_rx_bits_per_word32	32	uint32_t guaranteed size	C99 Sec.7.18.1.1 exact-width types

Constant	Value	Rationale	Standard Reference
k_rx_bits_per_word64	64	uint64_t guaranteed size	C99 Sec.7.18.1.1 exact-width types

Historical Note:

While some legacy systems used 7-bit bytes (e.g., CDC 6600) or 9-bit bytes (PDP-10), **all modern systems use 8-bit bytes** as mandated by POSIX.1-2008 and C99. The RX72N CHAR_BIT is guaranteed to be 8.

Usage Example: Endianness Conversion

```
#include "rx_bit_constants.h"

// Convert little-endian uint32_t to big-endian byte array
void uint32_to_big_endian(uint32_t value, uint8_t* buffer) {
  buffer[0] = (value » (k_rx_bits_per_byte * 3)) & 0xFF;  // MSB
  buffer[1] = (value » (k_rx_bits_per_byte * 2)) & 0xFF;
  buffer[2] = (value » (k_rx_bits_per_byte * 1)) & 0xFF;
  buffer[3] = (value » (k_rx_bits_per_byte * 0)) & 0xFF;  // LSB
}

// Convert big-endian byte array to little-endian uint32_t
uint32_t big_endian_to_uint32(const uint8_t* buffer) {
  return ((uint32_t)buffer[0] « (k_rx_bits_per_byte * 3)) |  // MSB
         ((uint32_t)buffer[1] « (k_rx_bits_per_byte * 2)) |
         ((uint32_t)buffer[2] « (k_rx_bits_per_byte * 1)) |
         ((uint32_t)buffer[3] « (k_rx_bits_per_byte * 0));   // LSB
}
```

See also

rx_bit_shifts_t for corresponding shift values

k_rx_shift_byte equivalent to (1 ∗ k_rx_bits_per_byte)

k_rx_shift_word16 equivalent to (2 ∗ k_rx_bits_per_byte)

Since

Version 1.0.0

Enumerator

k_rx_bits_per_byte	Bits per byte. Universal constant (CHAR_BIT = 8 per C99). Used for byte extraction, buffer sizing, bit-to-byte conversion. Range: Always 8 on modern systems.
k_rx_bits_per_word16	Bits per 16-bit word (uint16_t). Guaranteed by C99 exact-width types. Used for protocol fields, endianness conversion, CRC-16 operations. Range: Always 16.
k_rx_bits_per_word32	Bits per 32-bit word (uint32_t). Guaranteed by C99 exact-width types. Used for CRC-32, hash values, register access, Protocol Buffer fixed32. Range: Always 32.
k_rx_bits_per_word64	Bits per 64-bit word (uint64_t). Guaranteed by C99 exact-width types. Used for timestamps, large counters, Protocol Buffer fixed64. Range: Always 64.

Definition at line 229 of file rx_bit_constants.h.

```
00229             : uint8_t {
00230   k_rx_bits_per_byte =
00231     8, /**< Bits per byte. Universal constant (CHAR_BIT = 8 per C99). Used for byte extraction, buffer sizing, bit-to-byte
        conversion. Range: Always 8 on modern systems. */
00232   k_rx_bits_per_word16 =
00233     16, /**< Bits per 16-bit word (uint16_t). Guaranteed by C99 exact-width types. Used for protocol fields, endianness
        conversion, CRC-16 operations. Range: Always 16. */
00234   k_rx_bits_per_word32 =
00235     32, /**< Bits per 32-bit word (uint32_t). Guaranteed by C99 exact-width types. Used for CRC-32, hash values, register
        access, Protocol Buffer fixed32. Range: Always 32. */
00236   k_rx_bits_per_word64 =
00237     64, /**< Bits per 64-bit word (uint64_t). Guaranteed by C99 exact-width types. Used for timestamps, large counters,
        Protocol Buffer fixed64. Range: Always 64. */
00238 } rx_bit_sizes_t;
```

7.2 rx_bit_constants.h

Go to the documentation of this file.

```
00001 /* lib/rx_core/inc/rx_bit_constants.h */
00002
00003 /**
00004  * @file rx_bit_constants.h
00005  * @brief Fundamental Bit and Byte Size Constants for Protocol and Data Manipulation
00006  *
00007  * @details
00008  * ## Overview
00009  * Centralized repository of typed enum constants for bit/byte sizes and shift operations
00010  * used throughout the STAR firmware. Provides self-documenting, type-safe constants for:
00011  * - Protocol encoding/decoding (Protocol Buffers, ASCII frames, CRC)
00012  * - Data serialization and deserialization
00013  * - Bit manipulation (extraction, packing, masking)
00014  * - Endianness conversion (big-endian <-> little-endian)
00015  * - Buffer size calculations
00016  *
00017  * **Design Philosophy:**
00018  * - **No magic numbers**: All numeric literals replaced with named typed enums (NASA Rule 8)
00019  * - **Type safety**: C23 typed enums with explicit underlying types (uint8_t)
00020  * - **Self-documenting**: Constants have descriptive names (k_rx_bits_per_byte vs. literal 8)
00021  * - **Searchable**: grep for "k_rx_shift_byte" finds all byte-shift operations
00022  * - **Hardware-agnostic**: These are universal constants, not hardware-specific
00023  *
00024  * ## Scope and Usage
00025  *
00026  * **INCLUDE in:**
00027  * - Protocol encoding/decoding modules (rx_frame, rx_nanopb, rx_crc)
00028  * - Communication layers (rx_spi_comm, rx_usb_comm, rx_comm_manager)
00029  * - Utility functions for data packing/unpacking
00030  * - Unit conversion functions
00031  *
00032  * **EXCLUDE from (use peripheral-specific headers instead):**
00033  * - Hardware register bit positions -> Use rx72n_*_regs.h (e.g., USBE.SYSCFG.DRPD = bit 5)
00034  * - Clock divider calculations -> Use rx72n_clock.h (e.g., PCLKA_DIV = 2)
00035  * - Peripheral-specific shifts -> Define in peripheral header (e.g., ADC channel shift)
00036  *
00037  * ## Common Usage Patterns
00038  *
00039  * ### 1. Bit-to-Byte Conversion (Ceiling Division)
00040  * @code{.c}
00041  * // Calculate bytes needed to store N bits (round up)
00042  * uint32_t bit_count = 100;  // 100 bits
00043  * uint32_t byte_count = (bit_count + k_rx_bits_per_byte - 1) / k_rx_bits_per_byte;
00044  * // Result: (100 + 8 - 1) / 8 = 107 / 8 = 13 bytes (ceiling division)
00045  * @endcode
00046  *
00047  * ### 2. Byte Extraction from Multi-Byte Word
00048  * @code{.c}
00049  * // Extract individual bytes from 32-bit word (little-endian)
00050  * uint32_t word = 0x12345678;
00051  * uint8_t byte0 = (word » (k_rx_shift_byte * 0)) & 0xFF;  // 0x78 (LSB)
00052  * uint8_t byte1 = (word » (k_rx_shift_byte * 1)) & 0xFF;  // 0x56
00053  * uint8_t byte2 = (word » (k_rx_shift_byte * 2)) & 0xFF;  // 0x34
00054  * uint8_t byte3 = (word » k_rx_shift_word24) & 0xFF;      // 0x12 (MSB)
00055  * @endcode
00056  *
00057  * ### 3. Big-Endian 16-bit Packing
00058  * @code{.c}
00059  * // Pack uint16_t into big-endian byte array
00060  * uint16_t value = 0xABCD;
00061  * uint8_t buffer[2];
00062  * buffer[0] = (value » k_rx_shift_byte) & 0xFF;  // 0xAB (high byte)
00063  * buffer[1] = (value » 0) & 0xFF;                // 0xCD (low byte)
00064  * @endcode
00065  *
00066  * ### 4. Buffer Size Calculation for Bit Fields
00067  * @code{.c}
00068  * // Allocate buffer for protocol with bit-stuffing
00069  * const uint32_t k_payload_bits = 512;  // 512 bits of data
00070  * const uint32_t k_overhead_bits = 32;  // CRC-32
00071  * const uint32_t k_total_bits = k_payload_bits + k_overhead_bits;
00072  *
00073  * // Calculate bytes needed (ceiling division)
00074  * const uint32_t k_buffer_size =
00075  *     (k_total_bits + k_rx_bits_per_byte - 1) / k_rx_bits_per_byte;
00076  * uint8_t buffer[k_buffer_size];  // 68 bytes
00077  * @endcode
00078  *
00079  * ## Hardware Requirements
00080  *
00081  * | Requirement | Value | Notes |
00082  * |-------------|-------|-------|
```

```
00083  * | **CPU** | Any (RX, ARM, x86) | Universal constants, platform-agnostic |
00084  * | **Compiler** | C23 or later | Requires typed enum support `: uint8_t` |
00085  * | **Dependencies** | stdint.h | For uint8_t underlying type |
00086  * | **RAM** | 0 bytes | Compile-time constants (no storage) |
00087  * | **Flash** | 0 bytes | Inlined by compiler, no code generated |
00088  *
00089  * @par NASA Power of 10 Compliance:
00090  * - **Rule 1 [YES]:** No goto, setjmp/longjmp, recursion (header-only, no control flow)
00091  * - **Rule 2 [N/A]:** No loops (header-only)
00092  * - **Rule 3 [YES]:** No dynamic allocation (compile-time constants)
00093  * - **Rule 4 [N/A]:** No functions (header-only)
00094  * - **Rule 5 [N/A]:** No functions (header-only)
00095  * - **Rule 6 [YES]:** Data at smallest scope (file-scoped enums)
00096  * - **Rule 7 [N/A]:** No functions (header-only)
00097  * - **Rule 8 [YES]:** No macros except include guards (uses typed enums instead)
00098  * - **Rule 9 [YES]:** No pointers (value types only)
00099  * - **Rule 10 [YES]:** Compiles with -Wall -Wextra -Werror
00100  *
00101  * @par SOLID Principles:
00102  * - **S (Single Responsibility):** Module provides ONLY universal bit/byte constants
00103  * - **O (Open/Closed):** Extensible (add new enums) without modifying existing code
00104  * - **L (Liskov Substitution):** N/A (no functions, only data)
00105  * - **I (Interface Segregation):** Minimal API (2 enums), each with focused purpose
00106  * - **D (Dependency Inversion):** No dependencies (standalone header)
00107  *
00108  * @par Module Dependencies:
00109  * ```
00110  * rx_bit_constants.h
00111  *   +--> stdint.h (uint8_t for underlying enum type)
00112  *
00113  * Used by:
00114  *   +--> lib/rx_frame/ (protocol framing, byte packing)
00115  *   +--> lib/rx_crc/ (CRC-32 byte-wise operations)
00116  *   +--> lib/rx_nanopb/ (Protocol Buffer serialization)
00117  *   +--> lib/rx_comm_manager/ (communication buffer management)
00118  *   +--> lib/rx_hal/src/uart.c (data serialization)
00119  * ```
00120  *
00121  * @author STAR Team
00122  * @date 2026-01-27
00123  * @copyright MIT License
00124  *
00125  * @see stdint.h for standard integer types
00126  * @see rx_gpio_constants.h for GPIO pin constants
00127  * @see rx_port_constants.h for port/pin mapping constants
00128  * @see rx_time_constants.h for time conversion constants
00129  *
00130  * @version 1.0.0
00131  * @since Version 1.0.0
00132  */
00133
00134 #pragma once
00135
00136 #include <stdint.h>
00137
00138 #ifdef __cplusplus
00139 extern "C" {
00140 #endif
00141
00142 /*
     ================================================================================
00143  * Bit and Byte Size Constants
00144  *
     ================================================================================
00145  */
00146
00147 /**
00148  * @enum rx_bit_sizes_t
00149  * @brief Fundamental bit and byte size constants for data type sizes
00150  *
00151  * @details
00152  * Defines universal constants for bit counts in standard data types (byte, word16, word32, word64).
00153  * These constants eliminate magic numbers in protocol encoding, buffer calculations, and bit
00154  * manipulation operations.
00155  *
00156  * **Underlying Type:** uint8_t (8-bit unsigned) - sufficient for all values (max = 64)
00157  * **Guaranteed Sizes:** All values are compile-time constants (no runtime overhead)
00158  *
00159  * ## Usage Patterns
00160  *
00161  * ### Ceiling Division for Bit-to-Byte Conversion
00162  * @code{.c}
00163  * // Calculate bytes needed for N bits (always round up)
00164  * uint32_t bit_count = 100;  // Example: 100 bits
00165  * uint32_t byte_count = (bit_count + k_rx_bits_per_byte - 1) / k_rx_bits_per_byte;
00166  * // Result: (100 + 7) / 8 = 13 bytes (ceiling of 100/8 = 12.5)
00167  * @endcode
```

```
00168  *
00169  * ### Byte Indexing in Multi-Byte Words
00170  * @code{.c}
00171  * // Extract byte N from 32-bit word
00172  * uint32_t word = 0xDEADBEEF;
00173  * uint8_t byte_index = 2;  // Want byte 2 (0xBE)
00174  * uint8_t byte = (word » (k_rx_bits_per_byte * byte_index)) & 0xFF;
00175  * // Result: (0xDEADBEEF » 16) & 0xFF = 0xBE
00176  * @endcode
00177  *
00178  * ### Protocol Buffer Sizing
00179  * @code{.c}
00180  * // Allocate buffer for protocol with CRC-32
00181  * const uint32_t k_payload_bytes = 64;
00182  * const uint32_t k_crc_bits = k_rx_bits_per_word32;  // CRC-32 = 32 bits
00183  * const uint32_t k_crc_bytes = k_crc_bits / k_rx_bits_per_byte;  // 4 bytes
00184  * const uint32_t k_total_bytes = k_payload_bytes + k_crc_bytes;  // 68 bytes
00185  * uint8_t buffer[k_total_bytes];
00186  * @endcode
00187  *
00188  * ## Value Justification Table
00189  *
00190  * | Constant | Value | Rationale | Standard Reference |
00191  * |----------|-------|-----------|--------------------|
00192  * | k_rx_bits_per_byte | 8 | Universal (ANSI, ISO/IEC, POSIX) | C99 Sec.3.6: CHAR_BIT = 8 |
00193  * | k_rx_bits_per_word16 | 16 | uint16_t guaranteed size | C99 Sec.7.18.1.1 exact-width types |
00194  * | k_rx_bits_per_word32 | 32 | uint32_t guaranteed size | C99 Sec.7.18.1.1 exact-width types |
00195  * | k_rx_bits_per_word64 | 64 | uint64_t guaranteed size | C99 Sec.7.18.1.1 exact-width types |
00196  *
00197  * @par Historical Note:
00198  * While some legacy systems used 7-bit bytes (e.g., CDC 6600) or 9-bit bytes (PDP-10),
00199  * **all modern systems use 8-bit bytes** as mandated by POSIX.1-2008 and C99. The RX72N
00200  * CHAR_BIT is guaranteed to be 8.
00201  *
00202  * @par Usage Example: Endianness Conversion
00203  * @code{.c}
00204  * #include "rx_bit_constants.h"
00205  *
00206  * // Convert little-endian uint32_t to big-endian byte array
00207  * void uint32_to_big_endian(uint32_t value, uint8_t* buffer) {
00208  *   buffer[0] = (value » (k_rx_bits_per_byte * 3)) & 0xFF;  // MSB
00209  *   buffer[1] = (value » (k_rx_bits_per_byte * 2)) & 0xFF;
00210  *   buffer[2] = (value » (k_rx_bits_per_byte * 1)) & 0xFF;
00211  *   buffer[3] = (value » (k_rx_bits_per_byte * 0)) & 0xFF;  // LSB
00212  * }
00213  *
00214  * // Convert big-endian byte array to little-endian uint32_t
00215  * uint32_t big_endian_to_uint32(const uint8_t* buffer) {
00216  *   return ((uint32_t)buffer[0] « (k_rx_bits_per_byte * 3)) |  // MSB
00217  *          ((uint32_t)buffer[1] « (k_rx_bits_per_byte * 2)) |
00218  *          ((uint32_t)buffer[2] « (k_rx_bits_per_byte * 1)) |
00219  *          ((uint32_t)buffer[3] « (k_rx_bits_per_byte * 0));   // LSB
00220  * }
00221  * @endcode
00222  *
00223  * @see rx_bit_shifts_t for corresponding shift values
00224  * @see k_rx_shift_byte equivalent to (1 * k_rx_bits_per_byte)
00225  * @see k_rx_shift_word16 equivalent to (2 * k_rx_bits_per_byte)
00226  *
00227  * @since Version 1.0.0
00228  */
00229 typedef enum : uint8_t {
00230   k_rx_bits_per_byte =
00231     8, /**< Bits per byte. Universal constant (CHAR_BIT = 8 per C99). Used for byte extraction, buffer sizing, bit-to-byte
       conversion. Range: Always 8 on modern systems. */
00232   k_rx_bits_per_word16 =
00233     16, /**< Bits per 16-bit word (uint16_t). Guaranteed by C99 exact-width types. Used for protocol fields, endianness
       conversion, CRC-16 operations. Range: Always 16. */
00234   k_rx_bits_per_word32 =
00235     32, /**< Bits per 32-bit word (uint32_t). Guaranteed by C99 exact-width types. Used for CRC-32, hash values, register
       access, Protocol Buffer fixed32. Range: Always 32. */
00236   k_rx_bits_per_word64 =
00237     64, /**< Bits per 64-bit word (uint64_t). Guaranteed by C99 exact-width types. Used for timestamps, large counters,
       Protocol Buffer fixed64. Range: Always 64. */
00238 } rx_bit_sizes_t;
00239
00240 /**
00241  * @enum rx_bit_shifts_t
00242  * @brief Bit shift constants for byte and word extraction from multi-byte values
00243  *
00244  * @details
00245  * Provides named constants for common bit-shift operations used in protocol encoding,
00246  * endianness conversion, and byte-wise data access. These constants make code self-
00247  * documenting and eliminate magic shift values.
00248  *
00249  * **Underlying Type:** uint8_t (8-bit unsigned) - sufficient for all values (max = 24)
00250  * **Relationship to rx_bit_sizes_t:**
```

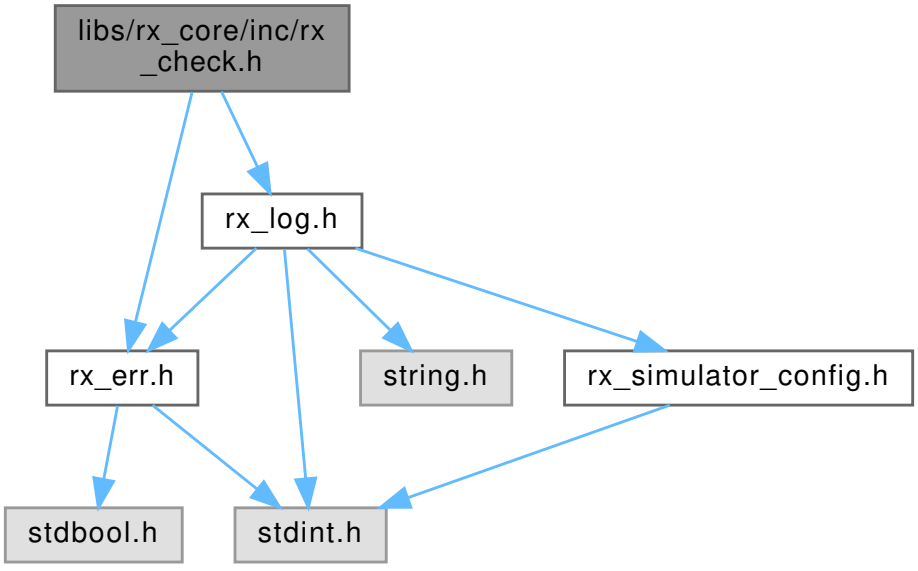
```
00251  * - k_rx_shift_byte = k_rx_bits_per_byte (both = 8)
00252  * - k_rx_shift_word16 = k_rx_bits_per_word16 (both = 16)
00253  * - k_rx_shift_word24 = 3 x k_rx_bits_per_byte (3 bytes = 24 bits)
00254  *
00255  * ## Usage Patterns
00256  *
00257  * ### Byte Extraction from 32-bit Word (Little-Endian)
00258  * @code{.c}
00259  * uint32_t word = 0x12345678;  // Little-endian representation
00260  *
00261  * // Extract individual bytes
00262  * uint8_t byte0 = (word » 0) & 0xFF;                // 0x78 (LSB, byte 0)
00263  * uint8_t byte1 = (word » k_rx_shift_byte) & 0xFF;   // 0x56 (byte 1)
00264  * uint8_t byte2 = (word » k_rx_shift_word16) & 0xFF; // 0x34 (byte 2)
00265  * uint8_t byte3 = (word » k_rx_shift_word24) & 0xFF; // 0x12 (MSB, byte 3)
00266  * @endcode
00267  *
00268  * ### Big-Endian 32-bit Packing
00269  * @code{.c}
00270  * // Pack uint32_t into big-endian byte array
00271  * uint32_t value = 0xAABBCCDD;
00272  * uint8_t buffer[4];
00273  *
00274  * buffer[0] = (value » k_rx_shift_word24) & 0xFF;  // 0xAA (MSB first)
00275  * buffer[1] = (value » k_rx_shift_word16) & 0xFF;  // 0xBB
00276  * buffer[2] = (value » k_rx_shift_byte) & 0xFF;    // 0xCC
00277  * buffer[3] = (value » 0) & 0xFF;                 // 0xDD (LSB last)
00278  * @endcode
00279  *
00280  * ### Big-Endian 16-bit Packing/Unpacking
00281  * @code{.c}
00282  * // Pack uint16_t -> big-endian buffer
00283  * uint16_t value = 0xABCD;
00284  * uint8_t buffer[2];
00285  * buffer[0] = (value » k_rx_shift_byte) & 0xFF;  // 0xAB (high byte)
00286  * buffer[1] = (value » 0) & 0xFF;                // 0xCD (low byte)
00287  *
00288  * // Unpack big-endian buffer -> uint16_t
00289  * uint16_t reconstructed = ((uint16_t)buffer[0] « k_rx_shift_byte) | buffer[1];
00290  * // Result: (0xAB « 8) | 0xCD = 0xABCD
00291  * @endcode
00292  *
00293  * ## Why Not Use rx_bit_sizes_t Directly?
00294  *
00295  * **Semantic Clarity:**
00296  * - `k_rx_shift_byte` **clearly indicates** "shift to access next byte"
00297  * - `k_rx_bits_per_byte` **ambiguously suggests** "number of bits in a byte"
00298  * - Using k_rx_shift_byte makes the **intent explicit** (shift operation)
00299  *
00300  * **Example:**
00301  * @code{.c}
00302  * // CLEAR INTENT: Using k_rx_shift_byte
00303  * uint8_t byte1 = (word » k_rx_shift_byte) & 0xFF;  // [OK] Obvious: shift by 1 byte
00304  *
00305  * // AMBIGUOUS INTENT: Using k_rx_bits_per_byte
00306  * uint8_t byte1 = (word » k_rx_bits_per_byte) & 0xFF;  // [X] Less clear: shifting? counting?
00307  * @endcode
00308  *
00309  * ## Value Justification Table
00310  *
00311  * | Constant | Value | Derivation | Use Case |
00312  * |----------|-------|------------|----------|
00313  * | k_rx_shift_byte | 8 | 1 byte x 8 bits | Access byte N in multi-byte word |
00314  * | k_rx_shift_word16 | 16 | 2 bytes x 8 bits | Access upper 16 bits of 32-bit word |
00315  * | k_rx_shift_word24 | 24 | 3 bytes x 8 bits | Access MSB (byte 3) of 32-bit word |
00316  *
00317  * @par Usage Example: CRC-32 Byte-Wise Processing
00318  * @code{.c}
00319  * #include "rx_bit_constants.h"
00320  *
00321  * // Extract bytes from CRC-32 for transmission (big-endian)
00322  * void crc32_to_bytes(uint32_t crc, uint8_t* buffer) {
00323  *   buffer[0] = (crc » k_rx_shift_word24) & 0xFF;  // Byte 3 (MSB)
00324  *   buffer[1] = (crc » k_rx_shift_word16) & 0xFF;  // Byte 2
00325  *   buffer[2] = (crc » k_rx_shift_byte) & 0xFF;    // Byte 1
00326  *   buffer[3] = (crc » 0) & 0xFF;                 // Byte 0 (LSB)
00327  * }
00328  *
00329  * // Reconstruct CRC-32 from big-endian byte array
00330  * uint32_t bytes_to_crc32(const uint8_t* buffer) {
00331  *   return ((uint32_t)buffer[0] « k_rx_shift_word24) |  // MSB
00332  *          ((uint32_t)buffer[1] « k_rx_shift_word16) |
00333  *          ((uint32_t)buffer[2] « k_rx_shift_byte) |
00334  *          ((uint32_t)buffer[3] « 0);                 // LSB
00335  * }
00336  * @endcode
00337  *
```

```
00338  * @par Usage Example: Protocol Field Extraction
00339  * @code{.c}
00340  * // Extract 16-bit field from Protocol Buffer varint
00341  * uint32_t varint = 0x12345678;
00342  * uint16_t field = (varint » k_rx_shift_word16) & 0xFFFF;
00343  * // Result: 0x1234 (upper 16 bits)
00344  * @endcode
00345  *
00346  * @see rx_bit_sizes_t for related bit count constants
00347  * @see k_rx_bits_per_byte equivalent value to k_rx_shift_byte (both = 8)
00348  * @see k_rx_bits_per_word16 equivalent value to k_rx_shift_word16 (both = 16)
00349  *
00350  * @since Version 1.0.0
00351  */
00352 typedef enum : uint8_t {
00353   k_rx_shift_byte =
00354     8, /**< Shift amount to access next byte (8 bits = 1 byte). Use for extracting byte N from multi-byte word. Example:
       byte1 = (word » k_rx_shift_byte) & 0xFF. Equivalent to k_rx_bits_per_byte but semantically clearer for shift
       operations. */
00355   k_rx_shift_word16 =
00356     16, /**< Shift amount to access upper 16 bits of 32-bit word (16 bits = 2 bytes). Use for extracting high word from
       dword. Example: high_word = (dword » k_rx_shift_word16) & 0xFFFF. Also used for big-endian 16-bit
       packing/unpacking. */
00357   k_rx_shift_word24 =
00358     24, /**< Shift amount to access byte 3 (MSB) of 32-bit word (24 bits = 3 bytes). Use for extracting most significant
       byte. Example: msb = (word » k_rx_shift_word24) & 0xFF. Critical for big-endian 32-bit encoding. */
00359 } rx_bit_shifts_t;
00360
00361 #ifdef __cplusplus
00362 }
00363 #endif
```

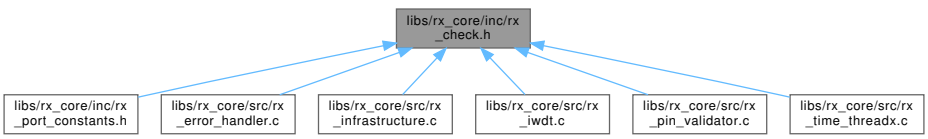
7.3 libs/rx_core/inc/rx_check.h File Reference

Validation and Error Checking Macros for RX72N Firmware.

#include "rx_err.h"
#include "rx_log.h"
Include dependency graph for rx_check.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define RX_ASSERT(condition, message)

  Runtime assertion with fatal error on failure (NASA Rule 5 compliance).
- #define RX_ERROR_CHECK(err)

  Fatal error checking - halt system if error code indicates failure.
- #define RX_ERROR_CHECK_WITHOUT_ABORT(err)

  Check error code and log on failure (non-fatal).
- #define RX_RETURN_ON_ERROR(err, tag, message)

  Return early on error.
- #define RX_RETURN_VOID_ON_ERROR(err, tag, message)

  Return void on error (for void functions).
- #define RX_RETURN_NULL_ON_ERROR(err, tag, message)

  Return NULL on error (for pointer-returning functions).
- #define RX_CHECK_NULL_PTR(ptr, tag, message)

  Check for null pointer and return error.
- #define RX_CHECK_RANGE(value, min, max, errcode)

  Check that a value is within an inclusive range.
- #define RX_CHECK_RANGE_TAG(value, min, max, errcode, tag)

  Check that a value is within an inclusive range with tag.
- #define RX_VALIDATE_PTR(ptr, tag, message)

  Validate pointer pre-condition and return error.
- #define RX_VALIDATE_INIT(initialized, tag, message)

  Validate initialization state pre-condition.

Functions

- static void internal_rx_fatal_error (const char *tag, const char *message, rx_err_t err)

  Halt execution with diagnostic error message (fatal error handler).

7.3.1 Detailed Description

Validation and Error Checking Macros for RX72N Firmware.

Comprehensive error checking and validation infrastructure for safety-critical embedded systems. Provides type-safe macros for parameter validation, error propagation, and assertion checking aligned with NASA Power of 10 Rule 5 (minimum 2 assertions per function).

7.3.1.1 Architecture Design

This module implements a layered error handling strategy:

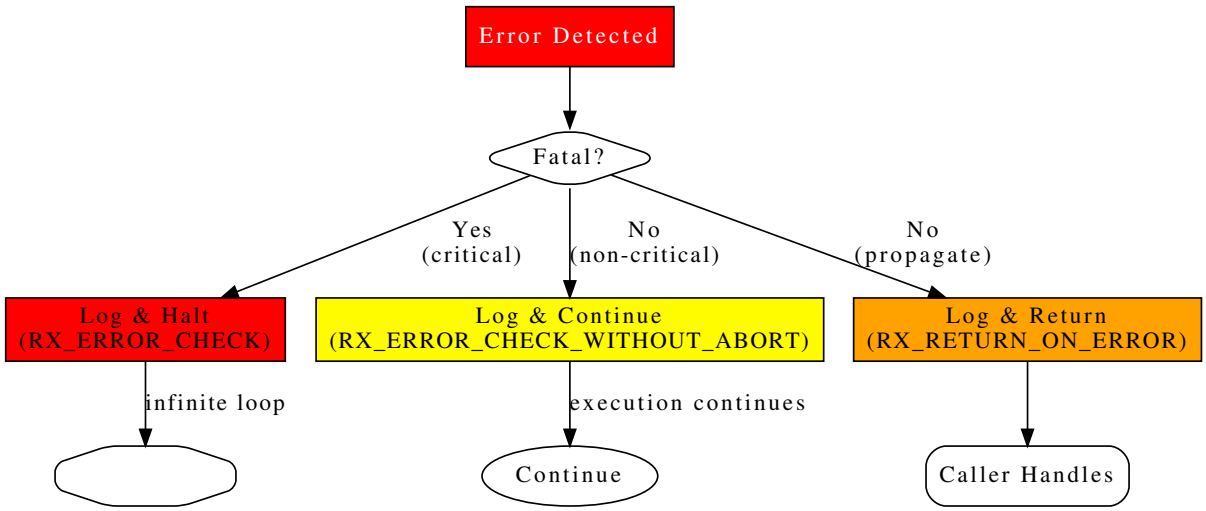
- **Fatal errors:** System halts with diagnostic output (RX_ERROR_CHECK, RX_ASSERT)
- **Propagating errors:** Early return with logging (RX_RETURN_ON_ERROR variants)
- **Non-fatal errors:** Log and continue (RX_ERROR_CHECK_WITHOUT_ABORT)
- **Precondition validation:** Input parameter checks (RX_CHECK_NULL_PTR, RX_CHECK_RANGE)

7.3.1.2 Error Handling Strategies

Macro	Fatal	Returns	Use Case
RX_ERROR_CHECK	[YES]	Never	Critical initialization errors

Macro	Fatal	Returns	Use Case
RX_ERROR_CHECK_WITHOUT_ABORT	[NO]	No	Optional feature failures
RX_RETURN_ON_ERROR	[NO]	rx_err_t	Error propagation in rx_err_t funcs
RX_RETURN_VOID_ON_ERROR	[NO]	void	Error handling in void functions
RX_RETURN_NULL_ON_ERROR	[NO]	NULL	Error handling in pointer functions
RX_CHECK_NULL_PTR	[NO]	rx_err_t	Null pointer precondition check
RX_CHECK_RANGE	[NO]	rx_err_t	Range validation
RX_ASSERT	[YES]	Never	Invariant and postcondition checking

7.3.1.3 Error Handling Flow



7.3.1.4 Design Rationale

Why macros instead of inline functions?

- Capture **FILE**, **LINE** for precise error location (future enhancement)
- Zero runtime overhead (no function call in release builds)
- Early return from enclosing function (not possible with inline functions)
- Conditional compilation based on build type (e.g., disable assertions in release)

Why do-while(0) wrapper?

- Safe in all contexts (if statements without braces)
- Requires semicolon after macro invocation
- Prevents multiple expansion bugs

Why separate return-type variants?

- Type safety: prevents returning error from void function
- Explicit intent: RX_RETURN_NULL_ON_ERROR clearly returnsnullptr
- Compiler optimization: separate code paths for different return types

7.3.1.5 Performance Characteristics

All macros expand to minimal code:

- **Success path:** 1-2 comparisons (2-4 cycles @ 240 MHz)
- **Error path:** Logging + action (varies, but typically $<$ 100 us for non-fatal)
- **Fatal error:** Logging + infinite loop (system halted)
- **Memory:** 0 bytes overhead (pure compile-time code generation)

7.3.1.6 Thread Safety

- **Validation macros:** Thread-safe (pure validation, no shared state)
- **Logging macros:** Thread-safe if rx_log is thread-safe
- **Fatal error:** Not thread-safe (intentional - halts entire system)

7.3.1.7 Usage Patterns

Pattern 1: Critical Initialization

```
void system_init(void) {
    rx_err_t err;

    // Critical hardware init - must succeed or halt
    err = init_clock();
    RX_ERROR_CHECK(err);  // Halts on failure

    err = init_gpio();
    RX_ERROR_CHECK(err);  // Halts on failure

    // System continues only if both succeeded
}
```

Pattern 2: Optional Feature Initialization

```
void init_optional_features(void) {
    rx_err_t err;

    // Optional feature - log error but continue
    err = init_debug_uart();
    RX_ERROR_CHECK_WITHOUT_ABORT(err);  // Logs, continues

    // System continues even if debug UART fails
}
```

Pattern 3: Error Propagation

```
rx_err_t init_subsystem(void) {
    rx_err_t err;

    // Propagate error to caller
    err = init_hardware();
    RX_RETURN_ON_ERROR(err, "SUBSYS", "Hardware init failed");

    err = init_software();
    RX_RETURN_ON_ERROR(err, "SUBSYS", "Software init failed");

    return k_rx_ok;
}
```

Pattern 4: NASA Rule 5 Compliance (Preconditions)

```
rx_err_t process_buffer(const uint8_t* data, size_t len) {
    // Precondition 1: Validate pointer (NASA Rule 5)
    RX_CHECK_NULL_PTR(data, "PROCESS", "Data buffer is nullptr");

    // Precondition 2: Validate size (NASA Rule 5)
    RX_CHECK_RANGE(len, 1, 256, k_rx_err_invalid_size);

    // Function body - preconditions satisfied
    // ...
    return k_rx_ok;
}
```

Pattern 5: Postcondition Validation

```
rx_err_t compute_result(float* output) {
    RX_CHECK_NULL_PTR(output, "COMPUTE", "Output pointer is nullptr");

    // Perform computation
    *output = complex_calculation();

    // Postcondition: result must be finite (NASA Rule 5)
    RX_ASSERT(isfinite(*output), "Result must be finite");

    return k_rx_ok;
}
```

Module Dependencies:

- rx_err.h: Error code definitions (rx_err_t, k_rx_ok, etc.)
- rx_log.h: Logging infrastructure (rx_log_error, uart_debug_puts)

See also

rx_err.h Error code definitions

rx_log.h Logging functions used by check macros

rx_error_handler.h High-level error handling infrastructure

docs/sections/06_nasa_power_of .tex NASA Rule 5 compliance

NASA Power of 10 Compliance:

- Rule 1: [YES] No goto, setjmp, recursion (macros expand to structured code)
- Rule 2: [YES] Infinite loop in fatal error is intentional (halt behavior)
- Rule 3: [YES] No dynamic allocation (stack-only variables)
- Rule 4: [YES] Macro expansions are small ($<$ 10 lines)
- Rule 5: [YES] **Explicitly designed to enforce Rule 5** (provide assertion macros)
- Rule 6: [YES] Minimal scope (do-while(0) block scope)
- Rule 7: [YES] All function calls checked by design (purpose of module)
- Rule 8: [YES] No raw macros for constants (uses rx_err_t typed enums)
- Rule 9: [YES] No function pointers
- Rule 10: [YES] Compiles with -Wall -Wextra -Werror

SOLID Principles:

- **S (Single Responsibility):** Only error checking and validation, no business logic
- **O (Open/Closed):** Extensible via new macros, existing macros closed for modification
- **L (Liskov Substitution):** Macros are transparent wrappers, don't change semantics
- **I (Interface Segregation):** Separate macros for different use cases (fatal, return, validate)
- **D (Dependency Inversion):** Depends on abstract rx_err_t, not concrete error values

Author

STAR Team

Date

2026-01-27

Version

1.0.0

Copyright

MIT License

Definition in file rx_check.h.

7.3.2 Macro Definition Documentation

7.3.2.1 RX_ASSERT

```
#define RX_ASSERT(
                    condition,
                    message)
```

Value:

```
do {                                                                    \
  if (!(condition)) {                                                     \
    internal_rx_fatal_error("ASSERT", message, k_rx_fail);                          \
  }                                                                     \
} while (0)
```

Runtime assertion with fatal error on failure (NASA Rule 5 compliance).

Verifies a boolean condition at runtime and halts execution if the condition is false. This macro is the primary mechanism for enforcing NASA Power of 10 Rule 5 (minimum 2 assertions per function) through precondition, postcondition, and invariant checking.

7.3.2.2 Algorithm

1. Evaluate condition expression
2. If condition is true: Continue execution (no overhead)
3. If condition is false: a. Call internal_rx_fatal_error() with "ASSERT" tag b. Log error message to debug UART c. Halt system (infinite loop with interrupts disabled)

7.3.2.3 Use Cases

Use Case	Example
Precondition (input)	RX_ASSERT(ptr != nullptr, "Input must be non-NULL")

Use Case	Example
Precondition (range)	RX_ASSERT(value <= MAX, "Value exceeds maximum")
Postcondition (output)	RX_ASSERT(result >= 0, "Result must be non-negative")
Invariant (state)	RX_ASSERT(count < MAX_COUNT, "Count overflow")
Loop bound verification	RX_ASSERT(i < ARRAY_SIZE, "Loop index overflow")

7.3.2.4 NASA Rule 5 Compliance Pattern

Every function should have minimum 2 assertions:

```
rx_err_t calculate(float* input, float* output) {
    // Precondition 1: Input pointer valid (NASA Rule 5)
    RX_ASSERT(input != nullptr, "Input pointer must not be NULL");

    // Precondition 2: Output pointer valid (NASA Rule 5)
    RX_ASSERT(output != nullptr, "Output pointer must not be NULL");

    // Perform calculation
    *output = *input * 2.0f;

    // Postcondition: Result is finite (NASA Rule 5)
    RX_ASSERT(isfinite(*output), "Output must be finite");

    return k_rx_ok;
}
```

Parameters

condition	Boolean expression to verify • Type: Any expression convertible to bool • Must evaluate to true for normal operation • Examples: ptr != nullptr, value < MAX, state == READY • Evaluated exactly once (no multiple evaluation bug) • Should not have side effects (evaluation may be removed in release builds)
message	Error message displayed on assertion failure • Type: const char* (string literal) • Must be compile-time constant string • Should describe what was expected, not what failed • Good: "Pointer must not be NULL", "Value must be in range [0, 100]" • Bad: "ptr is nullptr", "Invalid value" • Recommended length: < 60 characters for readability

Usage Example - Precondition Checking:

```
rx_err_t motor_set_velocity(motor_handle_t* motor, float velocity_mps) {
    // Precondition 1: Handle valid
    RX_ASSERT(motor != nullptr, "Motor handle must not be NULL");

    // Precondition 2: Velocity within physical limits
    RX_ASSERT(velocity_mps >= -2.5f && velocity_mps <= 2.5f,
```

```
            "Velocity must be in range [-2.5, 2.5] m/s");

    // Precondition 3: Motor initialized
    RX_ASSERT(motor->initialized, "Motor must be initialized");

    // Function body - all preconditions satisfied
    motor->target_velocity = velocity_mps;
    return k_rx_ok;
}
```

Usage Example - Postcondition Checking:

```
rx_err_t sensor_read(sensor_t* sensor, float* temperature_c) {
    RX_ASSERT(sensor != nullptr, "Sensor handle must not be NULL");
    RX_ASSERT(temperature_c != nullptr, "Output pointer must not be NULL");

    // Read sensor via I2C
    uint16_t raw_value = read_i2c_register(sensor->address, TEMP_REG);
    *temperature_c = (raw_value * 0.0625f) - 40.0f;  // DS18B20 conversion

    // Postcondition: Temperature in physically possible range
    RX_ASSERT(*temperature_c >= -55.0f && *temperature_c <= 125.0f,
            "Temperature must be in valid sensor range");

    return k_rx_ok;
}
```

Usage Example - Invariant Checking:

```
typedef struct {
    uint8_t head;
    uint8_t tail;
    uint8_t data[64];
} ring_buffer_t;

void ring_buffer_push(ring_buffer_t* rb, uint8_t value) {
    RX_ASSERT(rb != nullptr, "Ring buffer must not be NULL");

    // Invariant: head and tail always within bounds
    RX_ASSERT(rb->head < 64, "Head index must be < 64");
    RX_ASSERT(rb->tail < 64, "Tail index must be < 64");

    rb->data[rb->head] = value;
    rb->head = (rb->head + 1) % 64;

    // Invariant maintained after modification
    RX_ASSERT(rb->head < 64, "Head index must remain < 64");
}
```

Usage Example - Loop Bound Verification (NASA Rule 2):

```
typedef enum : uint8_t {
    k_max_iterations = 100  // Explicit loop bound (NASA Rule 2)
} loop_bounds_t;

void process_array(uint8_t* data, size_t len) {
    RX_ASSERT(data != nullptr, "Data array must not be NULL");
    RX_ASSERT(len <= k_max_iterations, "Array too large for fixed loop bound");

    for (uint8_t i = 0; i < k_max_iterations && i < len; i++) {
        // Invariant: i is always within array bounds
        RX_ASSERT(i < len, "Loop index must be < array length");
        data[i] = process_byte(data[i]);
    }
}
```

Conditional Compilation:

Assertions can be disabled in release builds for performance:

```
#ifdef NDEBUG
#define RX_ASSERT(condition, message) ((void)0)  // No-op in release
#else
#define RX_ASSERT(condition, message) do { ... } while (0)  // As shown above
#endif
```

WARNING: Disabling assertions removes safety checks. Only disable after extensive testing with assertions enabled.

Performance:

- Success path: 1 comparison + 1 branch (2-3 cycles @ 240 MHz)
- Failure path: Logs and halts (system terminates, performance irrelevant)
- Stack usage: 0 bytes (no additional stack beyond function call if failure)
- Code size: ∼8 bytes per assertion in release builds with optimizations

Note

Thread Safety: Condition check is atomic (single comparison)

Re-entrancy: Re-entrant (until failure, then system halts)

Memory: No dynamic allocation

Warning

Assertions halt the system - Use for programming errors and invariant violations, NOT for recoverable runtime errors.

Condition must not have side effects - If assertions are disabled, side effects disappear. Bad: RX_ASSERT(ptr++ != nullptr, "...") Good: ptr++; RX_ASSERT(ptr != nullptr, "...")

Attention

Use assertions for programming errors (bugs in code) and return error codes for runtime errors (invalid user input, hardware faults).

Assertions vs. Error Returns:

Condition Type	Use Assertion	Use Error Return
Null pointer from caller	YES	NO
Value out of documented range	YES	NO
Invalid state transition	YES	NO
Hardware fault	NO	YES
Communication timeout	NO	YES
Sensor out of range	NO	YES

Precondition

condition is a valid boolean expression

message is a non-NULL string literal (compile-time constant)

Postcondition

If condition is true: Execution continues normally

If condition is false: System halts permanently (requires reset)

Invariant

If condition is false, function never returns

Condition evaluated exactly once (no multiple evaluation)

See also

internal_rx_fatal_error() Function called on assertion failure

RX_ERROR_CHECK() Related macro for error code checking

RX_CHECK_NULL_PTR() Alternative for null pointer checking with error return

NASA Power of 10 Compliance:

- Rule 1: [YES] No goto, setjmp, recursion (structured if statement)
- Rule 2: [YES] N/A (no loops)
- Rule 3: [YES] No dynamic allocation
- Rule 4: [YES] Macro expansion is 3 lines
- Rule 5: [YES] **This macro implements Rule 5** (provides assertion mechanism)
- Rule 6: [YES] do-while(0) provides minimal scope
- Rule 7: [YES] N/A (macro itself, not function)
- Rule 8: [YES] Uses typed enum k_rx_fail
- Rule 9: [YES] No function pointers
- Rule 10: [YES] Compiles with -Wall -Wextra -Werror

Since

Version 1.0.0

Definition at line 638 of file rx_check.h.

```
00638 #define RX_ASSERT(condition, message)                                    \
00639   do {                                                                    \
00640     if (!(condition)) {                                                   \
00641       internal_rx_fatal_error("ASSERT", message, k_rx_fail);              \
00642     }                                                                     \
00643   } while (0)
```

Referenced by rx_pin_from_pin(), and rx_port_from_pin().

7.3.2.5 RX_CHECK_NULL_PTR

```
#define RX_CHECK_NULL_PTR(
                ptr,
                tag,
                message)
```

Value:

```
do {                                                                \
  if ((ptr) == nullptr) {                                             \
    rx_log_error(tag, message);                                         \
    return k_rx_err_null_ptr;                                           \
  }                                                                 \
} while (0)
```

Check for null pointer and return error.

Parameters

in	*ptr*	Pointer to check

in	*tag*	Component tag for logging
in	*message*	Error message for logging

If ptr is nullptr, logs the error and returns k_rx_err_null_ptr.

Example: rx_err_t process_data(const uint8_t* data) { RX_CHECK_NULL_PTR(data, "PROCESS", "Data pointer is nullptr"); // Only reached if data != nullptr return k_rx_ok; }

Definition at line 863 of file rx_check.h.

```
00863 #define RX_CHECK_NULL_PTR(ptr, tag, message)                                   \
00864   do {                                                                         \
00865     if ((ptr) == nullptr) {                                                    \
00866       rx_log_error(tag, message);                                              \
00867       return k_rx_err_null_ptr;                                                \
00868     }                                                                          \
00869   } while (0)
```

Referenced by error_handler_deinit(), error_handler_get_interface(), error_handler_init(), impl_clear_errors(), impl_report_error(), impl_reset_retry_counter(), pin_validator_deinit(), pin_validator_get_interface(), pin_validator_init(), and rx_time_threadx_get_interface().

7.3.2.6 RX_CHECK_RANGE

```
#define RX_CHECK_RANGE(
                value,
                min,
                max,
                errcode)
```

Value:

```
do {                                                                               \
  if (((value) < (min)) || ((value) > (max))) {                                    \
    rx_log_error("CHECK", "Range check failed");                                   \
    return (errcode);                                                              \
  }                                                                                \
} while (0)
```

Check that a value is within an inclusive range.

Parameters

in	*value*	Value to check
in	*min*	Minimum allowed value
in	*max*	Maximum allowed value
in	*errcode*	Error to return on failure

Definition at line 879 of file rx_check.h.

```
00879 #define RX_CHECK_RANGE(value, min, max, errcode)                               \
00880   do {                                                                         \
00881     if (((value) < (min)) || ((value) > (max))) {                              \
00882       rx_log_error("CHECK", "Range check failed");                             \
00883       return (errcode);                                                        \
00884     }                                                                          \
00885   } while (0)
```

7.3.2.7 RX_CHECK_RANGE_TAG

```
#define RX_CHECK_RANGE_TAG(
                value,
                min,
                max,
                errcode,
                tag)
```

Value:

```
do {                                                                    \
  (void)(min); /* Document minimum bound; explicit check needed if min > 0 */          \
  if ((value) > (max)) {                                                \
    rx_log_error(tag, "Range check failed");                                \
    return (errcode);                                                   \
  }                                                                     \
} while (0)
```

Check that a value is within an inclusive range with tag.

Note

Only checks upper bound to avoid -Wtype-limits warnings when min == 0 and value is unsigned (comparison is always false). For non-zero min bounds, add an explicit lower-bound check before this macro. The min parameter is documented but not checked.

Parameters

in	*value*	Value to check
in	*min*	Minimum allowed value (documented only; add explicit check if min > 0)
in	*max*	Maximum allowed value
in	*errcode*	Error to return on failure
in	*tag*	Component tag for logging

Definition at line 900 of file rx_check.h.

```
00900 #define RX_CHECK_RANGE_TAG(value, min, max, errcode, tag)                       \
00901   do {                                                                    \
00902     (void)(min); /* Document minimum bound; explicit check needed if min > 0 */          \
00903     if ((value) > (max)) {                                                \
00904       rx_log_error(tag, "Range check failed");                                \
00905       return (errcode);                                                   \
00906     }                                                                     \
00907   } while (0)
```

7.3.2.8 RX_ERROR_CHECK

```
#define RX_ERROR_CHECK(
                err)
```

Value:

```
do {                                                                    \
  rx_err_t err_rc_ = (err);                                                 \
  if (rx_err_is_error(err_rc_)) {                                           \
    internal_rx_fatal_error("ERROR_CHECK", "Fatal error detected", err_rc_);          \
  }                                                                     \
} while (0)
```

Fatal error checking - halt system if error code indicates failure.

Checks an rx_err_t error code and halts the system with diagnostics if the error is not k_rx_ok. Use this macro for critical operations where failure is unrecoverable and continuing execution would be unsafe.

7.3.2.9 When to Use

- **Critical initialization:** Hardware init, clock config, essential peripherals
- **Safety-critical resources:** Emergency stop system, watchdog, fault detection
- **Unrecoverable errors:** Memory corruption, invalid firmware state

7.3.2.10 When NOT to Use

- **Optional features:** Use RX_ERROR_CHECK_WITHOUT_ABORT instead
- **Recoverable errors:** Use RX_RETURN_ON_ERROR to propagate to caller
- **Expected failures:** Handle explicitly with if/switch statements

Parameters

err	Error code expression to check • Type: Any expression that evaluates to rx_err_t • Evaluated exactly once (safe for expressions with side effects) • If k_rx_ok (0): Continues execution silently • If any other value: Halts system with diagnostics

Usage Example - Critical Initialization:

```
void system_boot(void) {
    rx_err_t err;

    // Clock init is critical - must succeed
    err = init_system_clock();
    RX_ERROR_CHECK(err);  // Halts on failure

    // GPIO init is critical - must succeed
    err = init_gpio();
    RX_ERROR_CHECK(err);  // Halts on failure

    // Only reached if both succeeded
    uart_debug_puts("System boot successful\r\n");
}
```

Usage Example - Safety-Critical Resource:

```
void init_safety_systems(void) {
    rx_err_t err;

    // Emergency stop system must work
    err = init_emergency_stop();
    RX_ERROR_CHECK(err);  // Safety-critical, cannot proceed if failed

    // Watchdog must be enabled
    err = init_independent_watchdog();
    RX_ERROR_CHECK(err);  // System unusable without watchdog
}
```

Comparison to Alternatives:

Macro	Halts	Logs	Returns	Use Case
RX_ERROR_CHECK	[YES]	[YES]	Never	Critical init

Macro	Halts	Logs	Returns	Use Case
RX_ERROR_CHECK_WITHOUT_ABORT	[NO]	[YES]	No	Optional features
RX_RETURN_ON_ERROR	[NO]	[YES]	Error	Propagate to caller
if (err != k_rx_ok)	[NO]	[NO]	Varies	Custom handling

Warning

System halts permanently - Use only for truly unrecoverable errors

Cannot be caught or handled - No try/catch, no error recovery

Attention

Watchdog behavior: If enabled, watchdog will eventually reset system. If disabled, requires manual reset/power cycle.

See also

RX_ERROR_CHECK_WITHOUT_ABORT() Non-fatal version (logs, continues)

RX_RETURN_ON_ERROR() Error propagation version (returns to caller)

internal_rx_fatal_error() Underlying fatal error handler

Since

Version 1.0.0

Definition at line 731 of file rx_check.h.

```
00731 #define RX_ERROR_CHECK(err)                                                    \
00732   do {                                                                         \
00733     rx_err_t err_rc_ = (err);                                                  \
00734     if (rx_err_is_error(err_rc_)) {                                            \
00735       internal_rx_fatal_error("ERROR_CHECK", "Fatal error detected", err_rc_); \
00736     }                                                                          \
00737   } while (0)
```

7.3.2.11 RX_ERROR_CHECK_WITHOUT_ABORT

```
#define RX_ERROR_CHECK_WITHOUT_ABORT(
                err)
```

Value:

```
do {                                                  \
  rx_err_t err_rc_ = (err);                           \
  if (rx_err_is_error(err_rc_)) {                     \
    rx_log_error_val("ERROR_CHECK", "Error", err_rc_); \
  }                                                   \
} while (0)
```

Check error code and log on failure (non-fatal).

Parameters

in	*err*	Error code to check

If err is not k_rx_ok, logs the error but continues execution. Use this for non-critical errors where system can continue.

Example: rx_err_t err = optional_feature_init(); RX_ERROR_CHECK_WITHOUT_ABORT(err); // Logs but continues

Definition at line 751 of file rx_check.h.

```
00751 #define RX_ERROR_CHECK_WITHOUT_ABORT(err)                                      \
00752   do {                                                                         \
00753     rx_err_t err_rc_ = (err);                                                  \
00754     if (rx_err_is_error(err_rc_)) {                                            \
00755       rx_log_error_val("ERROR_CHECK", "Error", err_rc_);                       \
00756     }                                                                          \
00757   } while (0)
```

7.3.2.12 RX_RETURN_NULL_ON_ERROR

```
#define RX_RETURN_NULL_ON_ERROR(
                err,
                tag,
                message)
```

Value:

```
do {                                                                    \
  rx_err_t err_rc_ = (err);                                             \
  if (rx_err_is_error(err_rc_)) {                                       \
    rx_log_error(tag, message);                                         \
    rx_log_error_val(tag, "Error", err_rc_);                            \
    return nullptr;                                                     \
  }                                                                     \
} while (0)
```

Return NULL on error (for pointer-returning functions).

Parameters

in	*err*	Error code to check
in	*tag*	Component tag for logging
in	*message*	Error message for logging

If err is not k_rx_ok, logs the error and returns NULL from the function. Use this for functions that return pointers.

Example: void* create_object(void) { rx_err_t err = validate_preconditions(); RX_RETURN_NULL_ON_ERROR(err, "FACTORY", "Precondition check failed"); // Only reached if validation succeeded return allocate_object(); }

Definition at line 832 of file rx_check.h.

```
00832 #define RX_RETURN_NULL_ON_ERROR(err, tag, message)                      \
00833   do {                                                                  \
00834     rx_err_t err_rc_ = (err);                                           \
00835     if (rx_err_is_error(err_rc_)) {                                     \
00836       rx_log_error(tag, message);                                       \
00837       rx_log_error_val(tag, "Error", err_rc_);                          \
00838       return nullptr;                                                   \
00839     }                                                                   \
00840   } while (0)
```

7.3.2.13 RX_RETURN_ON_ERROR

```
#define RX_RETURN_ON_ERROR(
                err,
                tag,
                message)
```

Value:

```
do {                                                                    \
  rx_err_t err_rc_ = (err);                                             \
  if (rx_err_is_error(err_rc_)) {                                       \
    rx_log_error(tag, message);                                         \
    rx_log_error_val(tag, "Error", err_rc_);                            \
    return err_rc_;                                                     \
  }                                                                     \
} while (0)
```

Return early on error.

Parameters

in	*err*	Error code to check

in	*tag*	Component tag for logging
in	*message*	Error message for logging

If err is not k_rx_ok, logs the error and returns err from the function. Use this for functions that return rx_err_t.

Example: rx_err_t init_subsystem(void) { rx_err_t err = gpio_init(); RX_RETURN_ON_ERROR(err, "SUBSYS", "GPIO init failed"); // Only reached if gpio_init succeeded return k_rx_ok; }

Definition at line 777 of file rx_check.h.

```
00777 #define RX__RETURN__ON__ERROR(err, tag, message)                                \
00778   do {                                                                          \
00779     rx__err__t err__rc__ = (err);                                               \
00780     if (rx__err__is__error(err__rc__)) {                                        \
00781       rx__log__error(tag, message);                                             \
00782       rx__log__error__val(tag, "Error", err__rc__);                             \
00783       return err__rc__;                                                         \
00784     }                                                                           \
00785   } while (0)
```

7.3.2.14 RX_RETURN_VOID_ON_ERROR

```
#define RX__RETURN__VOID__ON__ERROR(
                err,
                tag,
                message)
```

Value:

```
do {                                                                    \
  rx__err__t err__rc__ = (err);                                         \
  if (rx__err__is__error(err__rc__)) {                                  \
    rx__log__error(tag, message);                                       \
    rx__log__error__val(tag, "Error", err__rc__);                       \
    return;                                                             \
  }                                                                     \
} while (0)
```

Return void on error (for void functions).

Parameters

in	*err*	Error code to check
in	*tag*	Component tag for logging
in	*message*	Error message for logging

If err is not k_rx_ok, logs the error and returns from the function. Use this for void functions that cannot return an error code.

Example: void configure_system(void) { rx_err_t err = gpio_init(); RX_RETURN_VOID_ON_ERROR(err, "SYSTEM", "GPIO init failed"); // Only reached if gpio_init succeeded }

Definition at line 804 of file rx_check.h.

```
00804 #define RX__RETURN__VOID__ON__ERROR(err, tag, message)                          \
00805   do {                                                                          \
00806     rx__err__t err__rc__ = (err);                                               \
00807     if (rx__err__is__error(err__rc__)) {                                        \
00808       rx__log__error(tag, message);                                             \
00809       rx__log__error__val(tag, "Error", err__rc__);                             \
00810       return;                                                                   \
00811     }                                                                           \
00812   } while (0)
```

7.3.2.15 RX_VALIDATE_INIT

```
#define RX_VALIDATE_INIT(
                    initialized,
                    tag,
                    message)
```

Value:

```
do {                                                                    \
  if (!(initialized)) {                                                 \
    rx_log_error(tag, message);                                             \
    return k_rx_err_invalid_state;                                          \
  }                                                                     \
} while (0)
```

Validate initialization state pre-condition.

Returns k_rx_err_invalid_state on failure.

Definition at line 921 of file rx_check.h.

```
00921 #define RX_VALIDATE_INIT(initialized, tag, message)                             \
00922   do {                                                                    \
00923     if (!(initialized)) {                                                 \
00924       rx_log_error(tag, message);                                             \
00925       return k_rx_err_invalid_state;                                          \
00926     }                                                                     \
00927   } while (0)
```

7.3.2.16 RX_VALIDATE_PTR

```
#define RX_VALIDATE_PTR(
                    ptr,
                    tag,
                    message)
```

Value:

```
RX_CHECK_NULL_PTR(ptr, tag, message)
```

Validate pointer pre-condition and return error.

Alias for RX_CHECK_NULL_PTR to standardize pre-condition naming.

Definition at line 914 of file rx_check.h.

7.3.3 Function Documentation

7.3.3.1 internal_rx_fatal_error()

```
void internal_rx_fatal_error (
                    const char * tag,
                    const char * message,
                    rx_err_t err)    [inline], [static]
```

Halt execution with diagnostic error message (fatal error handler).

Terminates firmware execution after logging detailed error diagnostics to debug UART. This is a non-returning function used for unrecoverable errors where continuing execution would be unsafe.

7.3.3.2 Algorithm

1. Output formatted error banner to debug UART
2. Display component tag, error message, and error code (hex)
3. Indicate system halted state
4. Disable all interrupts (global interrupt disable)
5. Enter infinite loop with CPU in low-power WAIT state

7.3.3.3 Output Format

```
========================================
FATAL ERROR
========================================
[TAG] Error message here
Error code: 0x00000XXX
========================================
System halted. Reset required.
========================================
```

7.3.3.4 Platform-Specific Behavior

Normal Build (Hardware Target):

- Disables interrupts via $\mathrm{clrpsw\ i}$ (clear PSW I-flag)
- Enters infinite loop with wait instruction
- CPU remains in low-power wait state until hardware reset
- Watchdog will eventually trigger reset if enabled

Unit Test Build (UNIT_TEST defined):

- Returns immediately without halting
- Allows test harness to continue and detect fatal error
- Enables testing of error paths without hanging test runner

Parameters

in	*tag*	Component tag for error identification • Type: const char$*$ (null-terminated string) • Must not be nullptr (undefined behavior if nullptr) • Typical values: ”MOTOR”, ”COMM”, ”INIT”, ”ASSERT”, etc. • Used to identify which subsystem detected the error • Recommended length: 3-8 characters for readability

in	*message*	Human-readable error description • Type: const char* (null-terminated string) • Must not be nullptr (undefined behavior if nullptr) • Should describe what failed and why • Example: "GPIO initialization failed", "Invalid motor ID" • Maximum practical length: ~80 characters (UART line width)
in	*err*	Error code indicating failure type • Type: rx_err_t (int32_t) • Typically non-zero (zero indicates success, illogical for fatal error) • Displayed in hexadecimal (8 digits, zero-padded) • Used for post-mortem debugging via UART log capture

Returns

This function never returns (except in UNIT_TEST builds)

Usage Example - From RX_ERROR_CHECK:

```
rx_err_t err = critical_init_function();
if (err != k_rx_ok) {
    internal_rx_fatal_error("INIT", "Critical init failed", err);
    // Never reached in normal builds
}
```

Usage Example - From RX_ASSERT:

```
if (!(ptr != nullptr)) {
    internal_rx_fatal_error("ASSERT", "Pointer must not be NULL", k_rx_fail);
    // Never reached in normal builds
}
```

Usage Example - Direct Call (Rare):

```
// Only use directly if macro wrappers don't fit use case
if (unrecoverable_condition()) {
    internal_rx_fatal_error("SAFETY", "Unrecoverable hardware fault", k_rx_err_hw_error);
}
```

Assembly Instruction Details:

clrpsw i - Clear Processor Status Word I-flag

- Disables all maskable interrupts globally
- Non-maskable interrupts (NMI) still active

- Prevents interrupt handlers from interfering with error state
- Ensures system remains in known halted state

wait - Wait for Interrupt

- Places CPU in low-power state
- CPU wakes on interrupt, but interrupts disabled -> immediately waits again
- Reduces power consumption during halt state
- Effective infinite loop with minimal power draw

Watchdog Behavior:

If watchdog (IWDT) is enabled:

- Watchdog will expire after timeout period (typically 128ms-2s)
- System will reset automatically
- Error message captured before reset (if UART log buffered)
- Allows recovery from fatal errors in deployed systems

If watchdog is disabled:

- System remains halted indefinitely
- Requires manual hardware reset or power cycle
- Useful during development for post-mortem debugging

Performance:

- Execution time: ~500 us (UART output dominates)
- Stack usage: ~16 bytes (local variables, return address)
- UART bandwidth: ~100 bytes @ 115200 baud = ~9ms transmission time
- **Not performance-critical:** Only called on fatal errors

Note

Thread Safety: Not thread-safe, but irrelevant (system halting)

Re-entrancy: Not reentrant (disables interrupts, infinite loop)

Memory: No dynamic allocation, stack-only locals

Warning

This function never returns in normal builds - Calling code after this function is unreachable. Compiler may optimize it away.

Do not call from interrupt context - Disabling interrupts in ISR can cause undefined behavior. Use error flags instead.

Attention

System is permanently halted - Only recovery is hardware reset. Ensure watchdog is enabled in production builds for automatic recovery.

UART output is blocking - If UART is not initialized or not connected, uart_debug_puts() may hang. Ensure UART is functional.

Precondition

tag != nullptr (undefined behavior if nullptr, will likely crash)

message != nullptr (undefined behavior if nullptr, will likely crash)

UART debug interface initialized (or uart_debug_puts() will hang)

Postcondition

Interrupts disabled globally (PSW.I = 0)

CPU in infinite wait loop (never returns)

Error diagnostics sent to debug UART

System requires hardware reset to recover

Invariant

In UNIT_TEST builds, function returns immediately (allows testing)

In normal builds, function never returns (infinite loop)

See also

RX_ERROR_CHECK() Macro that calls this function

RX_ASSERT() Macro that calls this function

uart_debug_puts() UART output function used for diagnostics

uart_debug_puthex() UART hex output function

NASA Power of 10 Compliance:

- Rule 1: [YES] No goto, setjmp, recursion
- Rule 2: [WARN] Infinite loop is intentional (halt behavior, safety requirement)
- Rule 3: [YES] No dynamic allocation
- Rule 4: [YES] Function is ∼30 lines (compact error handler)
- Rule 5: [YES] 3 preconditions, 4 postconditions, 2 invariants
- Rule 6: [YES] Variables have minimal scope
- Rule 7: [YES] UART function calls assumed to succeed (no error checking needed)
- Rule 8: [YES] Uses typed enum for error codes
- Rule 9: [YES] No function pointers
- Rule 10: [YES] Compiles with -Wall -Wextra -Werror

Since

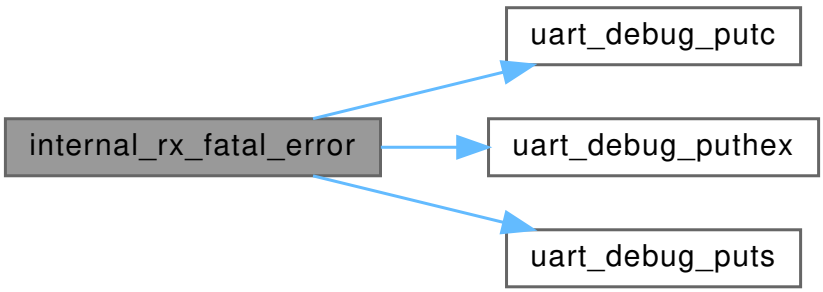
Version 1.0.0

Definition at line 390 of file rx_check.h.

```
00391 {
00392   uart_debug_puts("\r\n");
00393   uart_debug_puts("========================================\r\n");
00394   uart_debug_puts("FATAL ERROR\r\n");
00395   uart_debug_puts("========================================\r\n");
00396   uart_debug_putc('[');
00397   uart_debug_puts(tag);
00398   uart_debug_puts("] ");
00399   uart_debug_puts(message);
00400   uart_debug_puts("\r\n");
00401   uart_debug_puts("Error code: ");
00402   uart_debug_puthex((uint32_t)err, 8);
00403   uart_debug_puts("\r\n");
00404   uart_debug_puts("========================================\r\n");
00405   uart_debug_puts("System halted. Reset required.\r\n");
00406   uart_debug_puts("========================================\r\n");
00407
00408   /* Disable interrupts and halt */
00409 #ifndef UNIT_TEST
00410   __asm__ volatile("clrpsw i");
00411   while (1) {
00412     __asm__ volatile("wait");
00413   }
00414 #else
00415   return;
00416 #endif
00417 }
```

References uart_debug_putc(), uart_debug_puthex(), and uart_debug_puts().

Here is the call graph for this function:



7.4 rx_check.h

Go to the documentation of this file.

```
00001 /* lib/rx_core/inc/rx_check.h */
00002
00003 /**
00004  * @file rx_check.h
00005  * @brief Validation and Error Checking Macros for RX72N Firmware
00006  *
00007  * @details
00008  * Comprehensive error checking and validation infrastructure for safety-critical
00009  * embedded systems. Provides type-safe macros for parameter validation, error
00010  * propagation, and assertion checking aligned with NASA Power of 10 Rule 5
00011  * (minimum 2 assertions per function).
00012  *
00013  * ## Architecture Design
00014  *
00015  * This module implements a layered error handling strategy:
00016  * - **Fatal errors:** System halts with diagnostic output (RX_ERROR_CHECK, RX_ASSERT)
00017  * - **Propagating errors:** Early return with logging (RX_RETURN_ON_ERROR variants)
00018  * - **Non-fatal errors:** Log and continue (RX_ERROR_CHECK_WITHOUT_ABORT)
00019  * - **Precondition validation:** Input parameter checks (RX_CHECK_NULL_PTR, RX_CHECK_RANGE)
```

```
00020  *
00021  * ## Error Handling Strategies
00022  *
00023  * | Macro                          | Fatal | Returns | Use Case                                |
00024  * |--------------------------------|-------|---------|-----------------------------------------|
00025  * | RX_ERROR_CHECK                 | [YES]     | Never   | Critical initialization errors          |
00026  * | RX_ERROR_CHECK_WITHOUT_ABORT   | [NO]      | No      | Optional feature failures               |
00027  * | RX_RETURN_ON_ERROR             | [NO]      | rx_err_t| Error propagation in rx_err_t funcs     |
00028  * | RX_RETURN_VOID_ON_ERROR        | [NO]      | void    | Error handling in void functions        |
00029  * | RX_RETURN_NULL_ON_ERROR        | [NO]      | NULL    | Error handling in pointer functions     |
00030  * | RX_CHECK_NULL_PTR              | [NO]      | rx_err_t| Null pointer precondition check         |
00031  * | RX_CHECK_RANGE                 | [NO]      | rx_err_t| Range validation                        |
00032  * | RX_ASSERT                      | [YES]     | Never   | Invariant and postcondition checking    |
00033  *
00034  * ## Error Handling Flow
00035  *
00036  * @dot
00037  * digraph error_handling {
00038  *   rankdir=TB;
00039  *   node [shape=box, style=rounded];
00040  *
00041  *   error [label="Error Detected", fillcolor=red, style=filled, fontcolor=white];
00042  *   fatal [label="Fatal?", shape=diamond];
00043  *   log_halt [label="Log & Halt\n(RX_ERROR_CHECK)", fillcolor=red, style=filled];
00044  *   log_continue [label="Log & Continue\n(RX_ERROR_CHECK_WITHOUT_ABORT)", fillcolor=yellow, style=filled];
00045  *   log_return [label="Log & Return\n(RX_RETURN_ON_ERROR)", fillcolor=orange, style=filled];
00046  *   caller [label="Caller Handles"];
00047  *
00048  *   error -> fatal;
00049  *   fatal -> log_halt [label="Yes\n(critical)"];
00050  *   fatal -> log_continue [label="No\n(non-critical)"];
00051  *   fatal -> log_return [label="No\n(propagate)"];
00052  *   log_halt -> halt [label="infinite loop"];
00053  *   log_continue -> continue [label="execution continues"];
00054  *   log_return -> caller;
00055  *
00056  *   halt [shape=octagon, label="HALTED", fillcolor=black, fontcolor=white];
00057  *   continue [shape=ellipse, label="Continue"];
00058  * }
00059  * @enddot
00060  *
00061  * ## Design Rationale
00062  *
00063  * **Why macros instead of inline functions?**
00064  * - Capture __FILE__, __LINE__ for precise error location (future enhancement)
00065  * - Zero runtime overhead (no function call in release builds)
00066  * - Early return from enclosing function (not possible with inline functions)
00067  * - Conditional compilation based on build type (e.g., disable assertions in release)
00068  *
00069  * **Why do-while(0) wrapper?**
00070  * - Safe in all contexts (if statements without braces)
00071  * - Requires semicolon after macro invocation
00072  * - Prevents multiple expansion bugs
00073  *
00074  * **Why separate return-type variants?**
00075  * - Type safety: prevents returning error from void function
00076  * - Explicit intent: RX_RETURN_NULL_ON_ERROR clearly returnsnullptr
00077  * - Compiler optimization: separate code paths for different return types
00078  *
00079  * ## Performance Characteristics
00080  *
00081  * All macros expand to minimal code:
00082  * - **Success path:** 1-2 comparisons (2-4 cycles @ 240 MHz)
00083  * - **Error path:** Logging + action (varies, but typically < 100 us for non-fatal)
00084  * - **Fatal error:** Logging + infinite loop (system halted)
00085  * - **Memory:** 0 bytes overhead (pure compile-time code generation)
00086  *
00087  * ## Thread Safety
00088  *
00089  * - **Validation macros:** Thread-safe (pure validation, no shared state)
00090  * - **Logging macros:** Thread-safe if rx_log is thread-safe
00091  * - **Fatal error:** Not thread-safe (intentional - halts entire system)
00092  *
00093  * ## Usage Patterns
00094  *
00095  * @par Pattern 1: Critical Initialization
00096  * @code{.c}
00097  * void system_init(void) {
00098  *     rx_err_t err;
00099  *
00100  *     // Critical hardware init - must succeed or halt
00101  *     err = init_clock();
00102  *     RX_ERROR_CHECK(err);  // Halts on failure
00103  *
00104  *     err = init_gpio();
00105  *     RX_ERROR_CHECK(err);  // Halts on failure
00106  *
```

```
00107 *     // System continues only if both succeeded
00108 * }
00109 * @endcode
00110 *
00111 * @par Pattern 2: Optional Feature Initialization
00112 * @code{.c}
00113 * void init_optional_features(void) {
00114 *     rx_err_t err;
00115 *
00116 *     // Optional feature - log error but continue
00117 *     err = init_debug_uart();
00118 *     RX_ERROR_CHECK_WITHOUT_ABORT(err);  // Logs, continues
00119 *
00120 *     // System continues even if debug UART fails
00121 * }
00122 * @endcode
00123 *
00124 * @par Pattern 3: Error Propagation
00125 * @code{.c}
00126 * rx_err_t init_subsystem(void) {
00127 *     rx_err_t err;
00128 *
00129 *     // Propagate error to caller
00130 *     err = init_hardware();
00131 *     RX_RETURN_ON_ERROR(err, "SUBSYS", "Hardware init failed");
00132 *
00133 *     err = init_software();
00134 *     RX_RETURN_ON_ERROR(err, "SUBSYS", "Software init failed");
00135 *
00136 *     return k_rx_ok;
00137 * }
00138 * @endcode
00139 *
00140 * @par Pattern 4: NASA Rule 5 Compliance (Preconditions)
00141 * @code{.c}
00142 * rx_err_t process_buffer(const uint8_t* data, size_t len) {
00143 *     // Precondition 1: Validate pointer (NASA Rule 5)
00144 *     RX_CHECK_NULL_PTR(data, "PROCESS", "Data buffer is nullptr");
00145 *
00146 *     // Precondition 2: Validate size (NASA Rule 5)
00147 *     RX_CHECK_RANGE(len, 1, 256, k_rx_err_invalid_size);
00148 *
00149 *     // Function body - preconditions satisfied
00150 *     // ...
00151 *     return k_rx_ok;
00152 * }
00153 * @endcode
00154 *
00155 * @par Pattern 5: Postcondition Validation
00156 * @code{.c}
00157 * rx_err_t compute_result(float* output) {
00158 *     RX_CHECK_NULL_PTR(output, "COMPUTE", "Output pointer is nullptr");
00159 *
00160 *     // Perform computation
00161 *     *output = complex_calculation();
00162 *
00163 *     // Postcondition: result must be finite (NASA Rule 5)
00164 *     RX_ASSERT(isfinite(*output), "Result must be finite");
00165 *
00166 *     return k_rx_ok;
00167 * }
00168 * @endcode
00169 *
00170 * @par Module Dependencies:
00171 * - rx_err.h: Error code definitions (rx_err_t, k_rx_ok, etc.)
00172 * - rx_log.h: Logging infrastructure (rx_log_error, uart_debug_puts)
00173 *
00174 * @see rx_err.h Error code definitions
00175 * @see rx_log.h Logging functions used by check macros
00176 * @see rx_error_handler.h High-level error handling infrastructure
00177 * @see docs/sections/06_nasa_power_of .tex NASA Rule 5 compliance
00178 *
00179 * @par NASA Power of 10 Compliance:
00180 * - Rule 1: [YES] No goto, setjmp, recursion (macros expand to structured code)
00181 * - Rule 2: [YES] Infinite loop in fatal error is intentional (halt behavior)
00182 * - Rule 3: [YES] No dynamic allocation (stack-only variables)
00183 * - Rule 4: [YES] Macro expansions are small (< 10 lines)
00184 * - Rule 5: [YES] **Explicitly designed to enforce Rule 5** (provide assertion macros)
00185 * - Rule 6: [YES] Minimal scope (do-while(0) block scope)
00186 * - Rule 7: [YES] All function calls checked by design (purpose of module)
00187 * - Rule 8: [YES] No raw macros for constants (uses rx_err_t typed enums)
00188 * - Rule 9: [YES] No function pointers
00189 * - Rule 10: [YES] Compiles with -Wall -Wextra -Werror
00190 *
00191 * @par SOLID Principles:
00192 * - **S (Single Responsibility):** Only error checking and validation, no business logic
00193 * - **O (Open/Closed):** Extensible via new macros, existing macros closed for modification
```

```
00194  * - **L (Liskov Substitution):** Macros are transparent wrappers, don't change semantics
00195  * - **I (Interface Segregation):** Separate macros for different use cases (fatal, return, validate)
00196  * - **D (Dependency Inversion):** Depends on abstract rx_err_t, not concrete error values
00197  *
00198  * @author STAR Team
00199  * @date 2026-01-27
00200  * @version 1.0.0
00201  * @copyright MIT License
00202  */
00203
00204 #pragma once
00205
00206 #include "rx_err.h"
00207 #include "rx_log.h"
00208
00209 #ifdef __cplusplus
00210 extern "C" {
00211 #endif
00212
00213 /*
     ================================================================================
00214  * Fatal Error Handling
00215  *
     ================================================================================
00216  */
00217
00218 /**
00219  * @brief Halt execution with diagnostic error message (fatal error handler)
00220  *
00221  * @details
00222  * Terminates firmware execution after logging detailed error diagnostics to
00223  * debug UART. This is a non-returning function used for unrecoverable errors
00224  * where continuing execution would be unsafe.
00225  *
00226  * ## Algorithm
00227  *
00228  * 1. Output formatted error banner to debug UART
00229  * 2. Display component tag, error message, and error code (hex)
00230  * 3. Indicate system halted state
00231  * 4. Disable all interrupts (global interrupt disable)
00232  * 5. Enter infinite loop with CPU in low-power WAIT state
00233  *
00234  * ## Output Format
00235  *
00236  * ```
00237  * ========================================
00238  * FATAL ERROR
00239  * ========================================
00240  * [TAG] Error message here
00241  * Error code: 0x00000XXX
00242  * ========================================
00243  * System halted. Reset required.
00244  * ========================================
00245  * ```
00246  *
00247  * ## Platform-Specific Behavior
00248  *
00249  * **Normal Build (Hardware Target):**
00250  * - Disables interrupts via `clrpsw i` (clear PSW I-flag)
00251  * - Enters infinite loop with `wait` instruction
00252  * - CPU remains in low-power wait state until hardware reset
00253  * - Watchdog will eventually trigger reset if enabled
00254  *
00255  * **Unit Test Build (UNIT_TEST defined):**
00256  * - Returns immediately without halting
00257  * - Allows test harness to continue and detect fatal error
00258  * - Enables testing of error paths without hanging test runner
00259  *
00260  * @param[in] tag Component tag for error identification
00261  *   - Type: const char* (null-terminated string)
00262  *   - Must not be nullptr (undefined behavior if nullptr)
00263  *   - Typical values: "MOTOR", "COMM", "INIT", "ASSERT", etc.
00264  *   - Used to identify which subsystem detected the error
00265  *   - Recommended length: 3-8 characters for readability
00266  *
00267  * @param[in] message Human-readable error description
00268  *   - Type: const char* (null-terminated string)
00269  *   - Must not be nullptr (undefined behavior if nullptr)
00270  *   - Should describe what failed and why
00271  *   - Example: "GPIO initialization failed", "Invalid motor ID"
00272  *   - Maximum practical length: ~80 characters (UART line width)
00273  *
00274  * @param[in] err Error code indicating failure type
00275  *   - Type: rx_err_t (int32_t)
00276  *   - Typically non-zero (zero indicates success, illogical for fatal error)
00277  *   - Displayed in hexadecimal (8 digits, zero-padded)
00278  *   - Used for post-mortem debugging via UART log capture
```

```
00279  *
00280  * @return **This function never returns** (except in UNIT_TEST builds)
00281  *
00282  * @par Usage Example - From RX_ERROR_CHECK:
00283  * @code{.c}
00284  * rx_err_t err = critical_init_function();
00285  * if (err != k_rx_ok) {
00286  *     internal_rx_fatal_error("INIT", "Critical init failed", err);
00287  *     // Never reached in normal builds
00288  * }
00289  * @endcode
00290  *
00291  * @par Usage Example - From RX_ASSERT:
00292  * @code{.c}
00293  * if (!(ptr != nullptr)) {
00294  *     internal_rx_fatal_error("ASSERT", "Pointer must not be NULL", k_rx_fail);
00295  *     // Never reached in normal builds
00296  * }
00297  * @endcode
00298  *
00299  * @par Usage Example - Direct Call (Rare):
00300  * @code{.c}
00301  * // Only use directly if macro wrappers don't fit use case
00302  * if (unrecoverable_condition()) {
00303  *     internal_rx_fatal_error("SAFETY", "Unrecoverable hardware fault", k_rx_err_hw_error);
00304  * }
00305  * @endcode
00306  *
00307  * @par Assembly Instruction Details:
00308  *
00309  * **clrpsw i** - Clear Processor Status Word I-flag
00310  * - Disables all maskable interrupts globally
00311  * - Non-maskable interrupts (NMI) still active
00312  * - Prevents interrupt handlers from interfering with error state
00313  * - Ensures system remains in known halted state
00314  *
00315  * **wait** - Wait for Interrupt
00316  * - Places CPU in low-power state
00317  * - CPU wakes on interrupt, but interrupts disabled -> immediately waits again
00318  * - Reduces power consumption during halt state
00319  * - Effective infinite loop with minimal power draw
00320  *
00321  * @par Watchdog Behavior:
00322  *
00323  * If watchdog (IWDT) is enabled:
00324  * - Watchdog will expire after timeout period (typically 128ms-2s)
00325  * - System will reset automatically
00326  * - Error message captured before reset (if UART log buffered)
00327  * - Allows recovery from fatal errors in deployed systems
00328  *
00329  * If watchdog is disabled:
00330  * - System remains halted indefinitely
00331  * - Requires manual hardware reset or power cycle
00332  * - Useful during development for post-mortem debugging
00333  *
00334  * @par Performance:
00335  * - Execution time: ~500 us (UART output dominates)
00336  * - Stack usage: ~16 bytes (local variables, return address)
00337  * - UART bandwidth: ~100 bytes @ 115200 baud = ~9ms transmission time
00338  * - **Not performance-critical:** Only called on fatal errors
00339  *
00340  * @note Thread Safety: Not thread-safe, but irrelevant (system halting)
00341  * @note Re-entrancy: Not reentrant (disables interrupts, infinite loop)
00342  * @note Memory: No dynamic allocation, stack-only locals
00343  *
00344  * @warning **This function never returns in normal builds** - Calling code after
00345  *          this function is unreachable. Compiler may optimize it away.
00346  *
00347  * @warning **Do not call from interrupt context** - Disabling interrupts in ISR
00348  *          can cause undefined behavior. Use error flags instead.
00349  *
00350  * @attention **System is permanently halted** - Only recovery is hardware reset.
00351  *            Ensure watchdog is enabled in production builds for automatic recovery.
00352  *
00353  * @attention **UART output is blocking** - If UART is not initialized or not
00354  *            connected, uart_debug_puts() may hang. Ensure UART is functional.
00355  *
00356  * @pre tag != nullptr (undefined behavior if nullptr, will likely crash)
00357  * @pre message != nullptr (undefined behavior if nullptr, will likely crash)
00358  * @pre UART debug interface initialized (or uart_debug_puts() will hang)
00359  *
00360  * @post Interrupts disabled globally (PSW.I = 0)
00361  * @post CPU in infinite wait loop (never returns)
00362  * @post Error diagnostics sent to debug UART
00363  * @post System requires hardware reset to recover
00364  *
00365  * @invariant In UNIT_TEST builds, function returns immediately (allows testing)
```

```
00366  * @invariant In normal builds, function never returns (infinite loop)
00367  *
00368  * @see RX_ERROR_CHECK() Macro that calls this function
00369  * @see RX_ASSERT() Macro that calls this function
00370  * @see uart_debug_puts() UART output function used for diagnostics
00371  * @see uart_debug_puthex() UART hex output function
00372  *
00373  * @par NASA Power of 10 Compliance:
00374  * - Rule 1: [YES] No goto, setjmp, recursion
00375  * - Rule 2: [WARN] Infinite loop is intentional (halt behavior, safety requirement)
00376  * - Rule 3: [YES] No dynamic allocation
00377  * - Rule 4: [YES] Function is ~30 lines (compact error handler)
00378  * - Rule 5: [YES] 3 preconditions, 4 postconditions, 2 invariants
00379  * - Rule 6: [YES] Variables have minimal scope
00380  * - Rule 7: [YES] UART function calls assumed to succeed (no error checking needed)
00381  * - Rule 8: [YES] Uses typed enum for error codes
00382  * - Rule 9: [YES] No function pointers
00383  * - Rule 10: [YES] Compiles with -Wall -Wextra -Werror
00384  *
00385  * @since Version 1.0.0
00386  */
00387 #ifndef UNIT_TEST
00388 [[noreturn]]
00389 #endif
00390 static inline void internal_rx_fatal_error(const char* tag, const char* message, rx_err_t err)
00391 {
00392     uart_debug_puts("\r\n");
00393     uart_debug_puts("========================================\r\n");
00394     uart_debug_puts("FATAL ERROR\r\n");
00395     uart_debug_puts("========================================\r\n");
00396     uart_debug_putc('[');
00397     uart_debug_puts(tag);
00398     uart_debug_puts("] ");
00399     uart_debug_puts(message);
00400     uart_debug_puts("\r\n");
00401     uart_debug_puts("Error code: ");
00402     uart_debug_puthex((uint32_t)err, 8);
00403     uart_debug_puts("\r\n");
00404     uart_debug_puts("========================================\r\n");
00405     uart_debug_puts("System halted. Reset required.\r\n");
00406     uart_debug_puts("========================================\r\n");
00407
00408     /* Disable interrupts and halt */
00409 #ifndef UNIT_TEST
00410     __asm__ volatile("clrpsw i");
00411     while (1) {
00412         __asm__ volatile("wait");
00413     }
00414 #else
00415     return;
00416 #endif
00417 }
00418
00419 /*
    ============================================================================
00420  * Assertion Macros
00421  *
    ============================================================================
00422  */
00423
00424 /**
00425  * @def RX_ASSERT
00426  * @brief Runtime assertion with fatal error on failure (NASA Rule 5 compliance)
00427  *
00428  * @details
00429  * Verifies a boolean condition at runtime and halts execution if the condition
00430  * is false. This macro is the primary mechanism for enforcing NASA Power of 10
00431  * Rule 5 (minimum 2 assertions per function) through precondition, postcondition,
00432  * and invariant checking.
00433  *
00434  * ## Algorithm
00435  *
00436  * 1. Evaluate condition expression
00437  * 2. If condition is true: Continue execution (no overhead)
00438  * 3. If condition is false:
00439  *    a. Call internal_rx_fatal_error() with "ASSERT" tag
00440  *    b. Log error message to debug UART
00441  *    c. Halt system (infinite loop with interrupts disabled)
00442  *
00443  * ## Use Cases
00444  *
00445  * | Use Case                | Example                                          |
00446  * |-------------------------|--------------------------------------------------|
00447  * | Precondition (input)    | `RX_ASSERT(ptr != nullptr, "Input must be non-NULL")`|
00448  * | Precondition (range)    | `RX_ASSERT(value <= MAX, "Value exceeds maximum")`|
00449  * | Postcondition (output)  | `RX_ASSERT(result >= 0, "Result must be non-negative")`|
00450  * | Invariant (state)       | `RX_ASSERT(count < MAX_COUNT, "Count overflow")`|
```

```
00451  * | Loop bound verification   | `RX_ASSERT(i < ARRAY_SIZE, "Loop index overflow")`|
00452  *
00453  * ## NASA Rule 5 Compliance Pattern
00454  *
00455  * Every function should have minimum 2 assertions:
00456  * @code{.c}
00457  * rx_err_t calculate(float* input, float* output) {
00458  *     // Precondition 1: Input pointer valid (NASA Rule 5)
00459  *     RX_ASSERT(input != nullptr, "Input pointer must not be NULL");
00460  *
00461  *     // Precondition 2: Output pointer valid (NASA Rule 5)
00462  *     RX_ASSERT(output != nullptr, "Output pointer must not be NULL");
00463  *
00464  *     // Perform calculation
00465  *     *output = *input * 2.0f;
00466  *
00467  *     // Postcondition: Result is finite (NASA Rule 5)
00468  *     RX_ASSERT(isfinite(*output), "Output must be finite");
00469  *
00470  *     return k_rx_ok;
00471  * }
00472  * @endcode
00473  *
00474  * @param condition Boolean expression to verify
00475  *   - Type: Any expression convertible to bool
00476  *   - Must evaluate to true for normal operation
00477  *   - Examples: `ptr != nullptr`, `value < MAX`, `state == READY`
00478  *   - Evaluated exactly once (no multiple evaluation bug)
00479  *   - Should not have side effects (evaluation may be removed in release builds)
00480  *
00481  * @param message Error message displayed on assertion failure
00482  *   - Type: const char* (string literal)
00483  *   - Must be compile-time constant string
00484  *   - Should describe what was expected, not what failed
00485  *   - Good: "Pointer must not be NULL", "Value must be in range [0, 100]"
00486  *   - Bad: "ptr is nullptr", "Invalid value"
00487  *   - Recommended length: < 60 characters for readability
00488  *
00489  * @par Usage Example - Precondition Checking:
00490  * @code{.c}
00491  * rx_err_t motor_set_velocity(motor_handle_t* motor, float velocity_mps) {
00492  *     // Precondition 1: Handle valid
00493  *     RX_ASSERT(motor != nullptr, "Motor handle must not be NULL");
00494  *
00495  *     // Precondition 2: Velocity within physical limits
00496  *     RX_ASSERT(velocity_mps >= -2.5f && velocity_mps <= 2.5f,
00497  *               "Velocity must be in range [-2.5, 2.5] m/s");
00498  *
00499  *     // Precondition 3: Motor initialized
00500  *     RX_ASSERT(motor->initialized, "Motor must be initialized");
00501  *
00502  *     // Function body - all preconditions satisfied
00503  *     motor->target_velocity = velocity_mps;
00504  *     return k_rx_ok;
00505  * }
00506  * @endcode
00507  *
00508  * @par Usage Example - Postcondition Checking:
00509  * @code{.c}
00510  * rx_err_t sensor_read(sensor_t* sensor, float* temperature_c) {
00511  *     RX_ASSERT(sensor != nullptr, "Sensor handle must not be NULL");
00512  *     RX_ASSERT(temperature_c != nullptr, "Output pointer must not be NULL");
00513  *
00514  *     // Read sensor via I2C
00515  *     uint16_t raw_value = read_i2c_register(sensor->address, TEMP_REG);
00516  *     *temperature_c = (raw_value * 0.0625f) - 40.0f;  // DS18B20 conversion
00517  *
00518  *     // Postcondition: Temperature in physically possible range
00519  *     RX_ASSERT(*temperature_c >= -55.0f && *temperature_c <= 125.0f,
00520  *               "Temperature must be in valid sensor range");
00521  *
00522  *     return k_rx_ok;
00523  * }
00524  * @endcode
00525  *
00526  * @par Usage Example - Invariant Checking:
00527  * @code{.c}
00528  * typedef struct {
00529  *     uint8_t head;
00530  *     uint8_t tail;
00531  *     uint8_t data[64];
00532  * } ring_buffer_t;
00533  *
00534  * void ring_buffer_push(ring_buffer_t* rb, uint8_t value) {
00535  *     RX_ASSERT(rb != nullptr, "Ring buffer must not be NULL");
00536  *
00537  *     // Invariant: head and tail always within bounds
```

```
00538 *     RX_ASSERT(rb->head < 64, "Head index must be < 64");
00539 *     RX_ASSERT(rb->tail < 64, "Tail index must be < 64");
00540 *
00541 *     rb->data[rb->head] = value;
00542 *     rb->head = (rb->head + 1) % 64;
00543 *
00544 *     // Invariant maintained after modification
00545 *     RX_ASSERT(rb->head < 64, "Head index must remain < 64");
00546 * }
00547 * @endcode
00548 *
00549 * @par Usage Example - Loop Bound Verification (NASA Rule 2):
00550 * @code{.c}
00551 * typedef enum : uint8_t {
00552 *     k_max_iterations = 100  // Explicit loop bound (NASA Rule 2)
00553 * } loop_bounds_t;
00554 *
00555 * void process_array(uint8_t* data, size_t len) {
00556 *     RX_ASSERT(data != nullptr, "Data array must not be NULL");
00557 *     RX_ASSERT(len <= k_max_iterations, "Array too large for fixed loop bound");
00558 *
00559 *     for (uint8_t i = 0; i < k_max_iterations && i < len; i++) {
00560 *         // Invariant: i is always within array bounds
00561 *         RX_ASSERT(i < len, "Loop index must be < array length");
00562 *         data[i] = process_byte(data[i]);
00563 *     }
00564 * }
00565 * @endcode
00566 *
00567 * @par Conditional Compilation:
00568 *
00569 * Assertions can be disabled in release builds for performance:
00570 * @code{.c}
00571 * #ifdef NDEBUG
00572 * #define RX_ASSERT(condition, message) ((void)0)  // No-op in release
00573 * #else
00574 * #define RX_ASSERT(condition, message) do { ... } while (0)  // As shown above
00575 * #endif
00576 * @endcode
00577 *
00578 * **WARNING:** Disabling assertions removes safety checks. Only disable after
00579 * extensive testing with assertions enabled.
00580 *
00581 * @par Performance:
00582 * - Success path: 1 comparison + 1 branch (2-3 cycles @ 240 MHz)
00583 * - Failure path: Logs and halts (system terminates, performance irrelevant)
00584 * - Stack usage: 0 bytes (no additional stack beyond function call if failure)
00585 * - Code size: ~8 bytes per assertion in release builds with optimizations
00586 *
00587 * @note Thread Safety: Condition check is atomic (single comparison)
00588 * @note Re-entrancy: Re-entrant (until failure, then system halts)
00589 * @note Memory: No dynamic allocation
00590 *
00591 * @warning **Assertions halt the system** - Use for programming errors and
00592 *          invariant violations, NOT for recoverable runtime errors.
00593 *
00594 * @warning **Condition must not have side effects** - If assertions are disabled,
00595 *          side effects disappear. Bad: `RX_ASSERT(ptr++ != nullptr, "...")` Good: `ptr++; RX_ASSERT(ptr != nullptr,
    "...")`
00596 *
00597 * @attention **Use assertions for programming errors** (bugs in code) and
00598 *            return error codes for runtime errors (invalid user input, hardware faults).
00599 *
00600 * @par Assertions vs. Error Returns:
00601 *
00602 * | Condition Type           | Use Assertion    | Use Error Return |
00603 * |--------------------------|------------------|------------------|
00604 * | Null pointer from caller | YES              | NO               |
00605 * | Value out of documented range | YES         | NO               |
00606 * | Invalid state transition | YES              | NO               |
00607 * | Hardware fault           | NO               | YES              |
00608 * | Communication timeout    | NO               | YES              |
00609 * | Sensor out of range      | NO               | YES              |
00610 *
00611 * @pre condition is a valid boolean expression
00612 * @pre message is a non-NULL string literal (compile-time constant)
00613 *
00614 * @post If condition is true: Execution continues normally
00615 * @post If condition is false: System halts permanently (requires reset)
00616 *
00617 * @invariant If condition is false, function never returns
00618 * @invariant Condition evaluated exactly once (no multiple evaluation)
00619 *
00620 * @see internal_rx_fatal_error() Function called on assertion failure
00621 * @see RX_ERROR_CHECK() Related macro for error code checking
00622 * @see RX_CHECK_NULL_PTR() Alternative for null pointer checking with error return
00623 *
```

```
00624  * @par NASA Power of 10 Compliance:
00625  * - Rule 1: [YES] No goto, setjmp, recursion (structured if statement)
00626  * - Rule 2: [YES] N/A (no loops)
00627  * - Rule 3: [YES] No dynamic allocation
00628  * - Rule 4: [YES] Macro expansion is 3 lines
00629  * - Rule 5: [YES] **This macro implements Rule 5** (provides assertion mechanism)
00630  * - Rule 6: [YES] do-while(0) provides minimal scope
00631  * - Rule 7: [YES] N/A (macro itself, not function)
00632  * - Rule 8: [YES] Uses typed enum k_rx_fail
00633  * - Rule 9: [YES] No function pointers
00634  * - Rule 10: [YES] Compiles with -Wall -Wextra -Werror
00635  *
00636  * @since Version 1.0.0
00637  */
00638 #define RX_ASSERT(condition, message)                                          \
00639   do {                                                                         \
00640     if (!(condition)) {                                                        \
00641       internal_rx_fatal_error("ASSERT", message, k_rx_fail);                   \
00642     }                                                                          \
00643   } while (0)
00644
00645 /*
      ============================================================================
00646  * Error Checking Macros
00647  *
      ============================================================================
00648  */
00649
00650 /**
00651  * @def RX_ERROR_CHECK
00652  * @brief Fatal error checking - halt system if error code indicates failure
00653  *
00654  * @details
00655  * Checks an rx_err_t error code and halts the system with diagnostics if the
00656  * error is not k_rx_ok. Use this macro for critical operations where failure
00657  * is unrecoverable and continuing execution would be unsafe.
00658  *
00659  * ## When to Use
00660  *
00661  * - **Critical initialization:** Hardware init, clock config, essential peripherals
00662  * - **Safety-critical resources:** Emergency stop system, watchdog, fault detection
00663  * - **Unrecoverable errors:** Memory corruption, invalid firmware state
00664  *
00665  * ## When NOT to Use
00666  *
00667  * - **Optional features:** Use RX_ERROR_CHECK_WITHOUT_ABORT instead
00668  * - **Recoverable errors:** Use RX_RETURN_ON_ERROR to propagate to caller
00669  * - **Expected failures:** Handle explicitly with if/switch statements
00670  *
00671  * @param err Error code expression to check
00672  *   - Type: Any expression that evaluates to rx_err_t
00673  *   - Evaluated exactly once (safe for expressions with side effects)
00674  *   - If k_rx_ok (0): Continues execution silently
00675  *   - If any other value: Halts system with diagnostics
00676  *
00677  * @par Usage Example - Critical Initialization:
00678  * @code{.c}
00679  * void system_boot(void) {
00680  *     rx_err_t err;
00681  *
00682  *     // Clock init is critical - must succeed
00683  *     err = init_system_clock();
00684  *     RX_ERROR_CHECK(err);  // Halts on failure
00685  *
00686  *     // GPIO init is critical - must succeed
00687  *     err = init_gpio();
00688  *     RX_ERROR_CHECK(err);  // Halts on failure
00689  *
00690  *     // Only reached if both succeeded
00691  *     uart_debug_puts("System boot successful\r\n");
00692  * }
00693  * @endcode
00694  *
00695  * @par Usage Example - Safety-Critical Resource:
00696  * @code{.c}
00697  * void init_safety_systems(void) {
00698  *     rx_err_t err;
00699  *
00700  *     // Emergency stop system must work
00701  *     err = init_emergency_stop();
00702  *     RX_ERROR_CHECK(err);  // Safety-critical, cannot proceed if failed
00703  *
00704  *     // Watchdog must be enabled
00705  *     err = init_independent_watchdog();
00706  *     RX_ERROR_CHECK(err);  // System unusable without watchdog
00707  * }
00708  * @endcode
```

```
00709  *
00710  * @par Comparison to Alternatives:
00711  *
00712  * | Macro                          | Halts | Logs | Returns | Use Case               |
00713  * |--------------------------------|-------|------|---------|------------------------|
00714  * | RX_ERROR_CHECK                 | [YES]     | [YES]     | Never   | Critical init          |
00715  * | RX_ERROR_CHECK_WITHOUT_ABORT   | [NO]      | [YES]     | No      | Optional features      |
00716  * | RX_RETURN_ON_ERROR             | [NO]      | [YES]     | Error   | Propagate to caller    |
00717  * | if (err != k_rx_ok)            | [NO]      | [NO]      | Varies  | Custom handling        |
00718  *
00719  * @warning **System halts permanently** - Use only for truly unrecoverable errors
00720  * @warning **Cannot be caught or handled** - No try/catch, no error recovery
00721  *
00722  * @attention **Watchdog behavior:** If enabled, watchdog will eventually reset
00723  *            system. If disabled, requires manual reset/power cycle.
00724  *
00725  * @see RX_ERROR_CHECK_WITHOUT_ABORT() Non-fatal version (logs, continues)
00726  * @see RX_RETURN_ON_ERROR() Error propagation version (returns to caller)
00727  * @see internal_rx_fatal_error() Underlying fatal error handler
00728  *
00729  * @since Version 1.0.0
00730  */
00731 #define RX_ERROR_CHECK(err)                                                    \
00732   do {                                                                         \
00733     rx_err_t err_rc_ = (err);                                                  \
00734     if (rx_err_is_error(err_rc_)) {                                            \
00735       internal_rx_fatal_error("ERROR_CHECK", "Fatal error detected", err_rc_); \
00736     }                                                                          \
00737   } while (0)
00738
00739 /**
00740  * @brief Check error code and log on failure (non-fatal)
00741  *
00742  * @param[in] err Error code to check
00743  *
00744  * If err is not k_rx_ok, logs the error but continues execution.
00745  * Use this for non-critical errors where system can continue.
00746  *
00747  * Example:
00748  *   rx_err_t err = optional_feature_init();
00749  *   RX_ERROR_CHECK_WITHOUT_ABORT(err);  // Logs but continues
00750  */
00751 #define RX_ERROR_CHECK_WITHOUT_ABORT(err)                                      \
00752   do {                                                                         \
00753     rx_err_t err_rc_ = (err);                                                  \
00754     if (rx_err_is_error(err_rc_)) {                                            \
00755       rx_log_error_val("ERROR_CHECK", "Error", err_rc_);                       \
00756     }                                                                          \
00757   } while (0)
00758
00759 /**
00760  * @brief Return early on error
00761  *
00762  * @param[in] err Error code to check
00763  * @param[in] tag Component tag for logging
00764  * @param[in] message Error message for logging
00765  *
00766  * If err is not k_rx_ok, logs the error and returns err from the function.
00767  * Use this for functions that return rx_err_t.
00768  *
00769  * Example:
00770  *   rx_err_t init_subsystem(void) {
00771  *       rx_err_t err = gpio_init();
00772  *       RX_RETURN_ON_ERROR(err, "SUBSYS", "GPIO init failed");
00773  *       // Only reached if gpio_init succeeded
00774  *       return k_rx_ok;
00775  *   }
00776  */
00777 #define RX_RETURN_ON_ERROR(err, tag, message)                                  \
00778   do {                                                                         \
00779     rx_err_t err_rc_ = (err);                                                  \
00780     if (rx_err_is_error(err_rc_)) {                                            \
00781       rx_log_error(tag, message);                                              \
00782       rx_log_error_val(tag, "Error", err_rc_);                                 \
00783       return err_rc_;                                                          \
00784     }                                                                          \
00785   } while (0)
00786
00787 /**
00788  * @brief Return void on error (for void functions)
00789  *
00790  * @param[in] err Error code to check
00791  * @param[in] tag Component tag for logging
00792  * @param[in] message Error message for logging
00793  *
00794  * If err is not k_rx_ok, logs the error and returns from the function.
00795  * Use this for void functions that cannot return an error code.
```

```
00796  *
00797  * Example:
00798  *   void configure_system(void) {
00799  *       rx_err_t err = gpio_init();
00800  *       RX_RETURN_VOID_ON_ERROR(err, "SYSTEM", "GPIO init failed");
00801  *       // Only reached if gpio_init succeeded
00802  *   }
00803  */
00804 #define RX_RETURN_VOID_ON_ERROR(err, tag, message)                                  \
00805   do {                                                                              \
00806     rx_err_t err_rc_ = (err);                                                       \
00807     if (rx_err_is_error(err_rc_)) {                                                 \
00808       rx_log_error(tag, message);                                                   \
00809       rx_log_error_val(tag, "Error", err_rc_);                                      \
00810       return;                                                                       \
00811     }                                                                               \
00812   } while (0)
00813
00814 /**
00815  * @brief Return NULL on error (for pointer-returning functions)
00816  *
00817  * @param[in] err Error code to check
00818  * @param[in] tag Component tag for logging
00819  * @param[in] message Error message for logging
00820  *
00821  * If err is not k_rx_ok, logs the error and returns NULL from the function.
00822  * Use this for functions that return pointers.
00823  *
00824  * Example:
00825  *   void* create_object(void) {
00826  *       rx_err_t err = validate_preconditions();
00827  *       RX_RETURN_NULL_ON_ERROR(err, "FACTORY", "Precondition check failed");
00828  *       // Only reached if validation succeeded
00829  *       return allocate_object();
00830  *   }
00831  */
00832 #define RX_RETURN_NULL_ON_ERROR(err, tag, message)                                  \
00833   do {                                                                              \
00834     rx_err_t err_rc_ = (err);                                                       \
00835     if (rx_err_is_error(err_rc_)) {                                                 \
00836       rx_log_error(tag, message);                                                   \
00837       rx_log_error_val(tag, "Error", err_rc_);                                      \
00838       return nullptr;                                                               \
00839     }                                                                               \
00840   } while (0)
00841
00842 /*
      ============================================================================
00843  * Null Pointer Checking
00844  *
      ============================================================================
00845  */
00846
00847 /**
00848  * @brief Check for null pointer and return error
00849  *
00850  * @param[in] ptr Pointer to check
00851  * @param[in] tag Component tag for logging
00852  * @param[in] message Error message for logging
00853  *
00854  * If ptr is nullptr, logs the error and returns k_rx_err_null_ptr.
00855  *
00856  * Example:
00857  *   rx_err_t process_data(const uint8_t* data) {
00858  *       RX_CHECK_NULL_PTR(data, "PROCESS", "Data pointer is nullptr");
00859  *       // Only reached if data != nullptr
00860  *       return k_rx_ok;
00861  *   }
00862  */
00863 #define RX_CHECK_NULL_PTR(ptr, tag, message)                                        \
00864   do {                                                                              \
00865     if ((ptr) == nullptr) {                                                         \
00866       rx_log_error(tag, message);                                                   \
00867       return k_rx_err_null_ptr;                                                     \
00868     }                                                                               \
00869   } while (0)
00870
00871 /**
00872  * @brief Check that a value is within an inclusive range
00873  *
00874  * @param[in] value Value to check
00875  * @param[in] min Minimum allowed value
00876  * @param[in] max Maximum allowed value
00877  * @param[in] errcode Error to return on failure
00878  */
00879 #define RX_CHECK_RANGE(value, min, max, errcode)                                    \
00880   do {                                                                              \
```

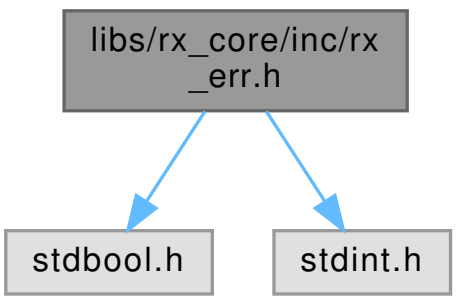
```
00881     if (((value) < (min)) || ((value) > (max))) {                                    \
00882       rx_log_error("CHECK", "Range check failed");                                    \
00883       return (errcode);                                                               \
00884     }                                                                                 \
00885   } while (0)
00886
00887 /**
00888  * @brief Check that a value is within an inclusive range with tag
00889  *
00890  * @note Only checks upper bound to avoid -Wtype-limits warnings when min == 0 and value is
00891  *       unsigned (comparison is always false). For non-zero min bounds, add an explicit
00892  *       lower-bound check before this macro. The min parameter is documented but not checked.
00893  *
00894  * @param[in] value Value to check
00895  * @param[in] min Minimum allowed value (documented only; add explicit check if min > 0)
00896  * @param[in] max Maximum allowed value
00897  * @param[in] errcode Error to return on failure
00898  * @param[in] tag Component tag for logging
00899  */
00900 #define RX_CHECK_RANGE_TAG(value, min, max, errcode, tag)                             \
00901   do {                                                                                \
00902     (void)(min); /* Document minimum bound; explicit check needed if min > 0 */     \
00903     if ((value) > (max)) {                                                            \
00904       rx_log_error(tag, "Range check failed");                                        \
00905       return (errcode);                                                               \
00906     }                                                                                 \
00907   } while (0)
00908
00909 /**
00910  * @brief Validate pointer pre-condition and return error
00911  *
00912  * Alias for RX_CHECK_NULL_PTR to standardize pre-condition naming.
00913  */
00914 #define RX_VALIDATE_PTR(ptr, tag, message) RX_CHECK_NULL_PTR(ptr, tag, message)
00915
00916 /**
00917  * @brief Validate initialization state pre-condition
00918  *
00919  * Returns k_rx_err_invalid_state on failure.
00920  */
00921 #define RX_VALIDATE_INIT(initialized, tag, message)                                   \
00922   do {                                                                                \
00923     if (!(initialized)) {                                                             \
00924       rx_log_error(tag, message);                                                     \
00925       return k_rx_err_invalid_state;                                                  \
00926     }                                                                                 \
00927   } while (0)
00928
00929 #ifdef __cplusplus
00930 }
00931 #endif
```

7.5 libs/rx_core/inc/rx_err.h File Reference

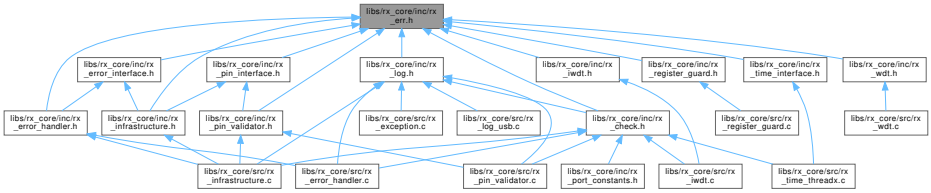
Error Code Definitions for RX72N Firmware.

```
#include <stdbool.h>
#include <stdint.h>
```

Include dependency graph for rx_err.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef int32_t rx_err_t

  Error code type for RX72N firmware.

Enumerations

- enum rx_err_codes_t : uint16_t {
  k_rx_ok = 0 , k_rx_fail = 0x101 , k_rx_err_no_mem = 0x102 , k_rx_err_invalid_arg = 0x103 ,
  k_rx_err_invalid_state = 0x104 , k_rx_err_invalid_size = 0x105 , k_rx_err_not_found = 0x106 ,
  k_rx_err_not_supported = 0x107 ,
  k_rx_err_timeout = 0x108 , k_rx_err_busy = 0x109 , k_rx_err_no_data = 0x10A , k_rx_err_would_block = 0x10B ,
  k_rx_err_exists = 0x10C , k_rx_err_empty = 0x10D , k_rx_err_cancelled = 0x10E , k_rx_err_not_initialized = 0x10F ,
  k_rx_err_estop = 0x110 , k_rx_err_hw_init_failed = 0x201 , k_rx_err_hw_not_ready = 0x202 , k_rx_err_hw_timeout = 0x203 ,
  k_rx_err_hw_error = 0x204 , k_rx_err_gpio_conflict = 0x205 , k_rx_err_gpio_invalid_port = 0x206 ,
  k_rx_err_gpio_invalid_pin = 0x207 ,
  k_rx_err_out_of_range = 0x208 , k_rx_err_hw_unmapped = 0x209 , k_rx_err_rtos_error = 0x301 ,
  k_rx_err_threadx = 0x301 ,
  k_rx_err_rtos_thread_create = 0x302 , k_rx_err_rtos_semaphore = 0x303 , k_rx_err_rtos_mutex = 0x304 ,
  k_rx_err_rtos_queue = 0x305 ,
  k_rx_err_rtos_timer = 0x306 , k_rx_err_comm_error = 0x401 , k_rx_err_spi_error = 0x402 , k_rx_err_uart_error = 0x403 ,
  k_rx_err_i2c_error = 0x404 , k_rx_err_crc_mismatch = 0x405 , k_rx_err_protocol_error = 0x406 ,
  k_rx_err_nack = 0x407 ,
  k_rx_err_conflict = 0x408 , k_rx_err_retry_limit = 0x409 , k_rx_err_validation_failed = 0x501 , k_rx_err_checksum_mismatch = 0x502 ,
  k_rx_err_range_check_failed = 0x503 , k_rx_err_null_ptr = 0x504 }

  Comprehensive error code enumeration for RX72N firmware.

Functions

- static rx_err_t rx_err_from_threadx (uint32_t tx_status)

  Convert ThreadX status code to rx_err_t.
- static bool rx_err_is_ok (rx_err_t err)

  Check if error code represents success.
- static bool rx_err_is_error (rx_err_t err)

  Check if error code represents failure.

7.5.1 Detailed Description

Error Code Definitions for RX72N Firmware.

Comprehensive error code system for the STAR RX72N motor controller firmware. Provides type-safe error handling with categorized error codes following C23 typed enum standards for embedded safety-critical systems.

7.5.1.1 Architecture Design

This module implements a centralized error handling system for safety-critical embedded requirements:

- **Type-safe:** Uses C23 typed enums (uint16_t underlying type)
- **Zero overhead:** Inline helper functions compile to single instructions
- **Extensible:** Category-based organization supports future error codes
- **Traceable:** Each error code maps to specific failure conditions

7.5.1.2 Error Code Organization

Error codes are organized into non-overlapping ranges to facilitate debugging and logging. The hex representation makes category identification trivial:

Range	Category	Purpose
0x000	Success	Operation completed successfully
0x100-0x1FF	Generic	General-purpose errors (null, timeout)
0x200-0x2FF	Hardware	Peripheral and GPIO errors
0x300-0x3FF	RTOS	ThreadX and synchronization errors
0x400-0x4FF	Communication	SPI, UART, I2C, protocol errors
0x500-0x5FF	Validation	Input validation and checksum errors

7.5.1.3 Design Rationale

Why signed int32_t for rx_err_t but unsigned uint16_t for enum?

- rx_err_t uses int32_t to allow negative error codes (future compatibility)
- Enum uses uint16_t to minimize memory usage (most values $<$ 0xFFFF)
- Implicit conversion is safe: uint16_t -$>$ int32_t always preserves value

Why category-based ranges instead of linear numbering?

- Enables fast error categorization in logging (e.g., ”0x2xx = hardware”)
- Supports future expansion within categories without renumbering
- Makes error codes human-readable in hex dumps and debuggers

Why inline helpers instead of macros?

- Type safety: compiler checks parameter types
- Debuggable: can step into functions, set breakpoints
- No macro pitfalls: no multiple evaluation, proper scoping

7.5.1.4 Performance Characteristics

All operations are zero-cost abstractions:

- rx_err_from_threadx(): Single comparison + conditional move (1-2 cycles)
- rx_err_is_ok(): Single comparison (1 cycle)
- rx_err_is_error(): Single comparison (1 cycle)

7.5.1.5 Memory Usage

- rx_err_t: 4 bytes per error variable
- rx_err_codes_t: 2 bytes per enum value
- Helper functions: 0 bytes (inlined by compiler at -O1+)

7.5.1.6 Thread Safety

All functions in this module are inherently thread-safe (pure functions). Error codes are read-only constants after compilation.

Module Dependencies:

- stdint.h: Fixed-width integer types
- stdbool.h: Boolean type for helper functions

See also

rx_check.h Validation macros that return rx_err_t codes

rx_error_handler.h Error logging and handling infrastructure

docs/sections/06_nasa_power_of .tex NASA compliance documentation

NASA Power of 10 Compliance:

- Rule 1: [OK] No goto, setjmp, recursion (pure constant definitions)
- Rule 2: [OK] N/A (no loops)
- Rule 3: [OK] No dynamic allocation (compile-time constants only)
- Rule 4: [OK] All functions $<$ 10 lines (simple inline helpers)
- Rule 5: [OK] Inline functions have implicit pre/post conditions
- Rule 6: [OK] All identifiers have minimal scope
- Rule 7: [OK] N/A (no function calls to check)
- Rule 8: [OK] Uses C23 typed enums (k_rx_err_codes_t : uint16_t)
- Rule 9: [OK] N/A (no function pointers)
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror

SOLID Principles:

- **S (Single Responsibility):** Only error code definitions, no handling logic
- **O (Open/Closed):** Extensible via new enum values, closed for modification
- **L (Liskov Substitution):** rx_err_t can substitute for int32_t in APIs
- **I (Interface Segregation):** Minimal interface (3 helper functions)
- **D (Dependency Inversion):** No dependencies on concrete implementations

Author

STAR Team

Date

2026-01-27

Version

1.0.0

Copyright

MIT License

Definition in file rx_err.h.

7.5.2 Typedef Documentation

7.5.2.1 rx_err_t

typedef int32_t rx_err_t

Error code type for RX72N firmware.

Signed 32-bit integer type used for all function return values that indicate success or failure. This type provides a consistent error handling interface across the entire firmware codebase.

7.5.2.2 Type Selection Rationale

Why int32_t instead of enum?

- Allows future negative error codes (e.g., -1 for EOF-like conditions)
- Compatible with standard POSIX error conventions
- Sufficient range for all error categories (only using 0x000-0x5FF currently)
- Efficient on 32-bit ARM Cortex-R4 (native register width)

Why signed instead of unsigned?

- Enables use of negative values for special conditions
- Matches C standard library conventions (errno, etc.)
- Prevents accidental unsigned wraparound bugs

7.5.2.3 Value Semantics

- **0 (k_rx_ok):** Success - operation completed as requested
- **Non-zero:** Error - operation failed, consult specific error code
- **Positive values:** Standard error codes (0x100-0x5FF)
- **Negative values:** Reserved for future use

7.5.2.4 Usage Pattern

Standard Error Checking:

```
rx_err_t err = some_function();
if (err != k_rx_ok) {
    // Handle error
    rx_log_error("TAG", "Operation failed: 0x%X", err);
    return err;  // Propagate error up call stack
}
```

Early Return Pattern:

```
rx_err_t init_subsystem(void) {
    rx_err_t err;

    err = init_hardware();
    if (err != k_rx_ok) return err;

    err = init_rtos();
    if (err != k_rx_ok) return err;

    return k_rx_ok;
}
```

Switch-Based Error Handling:

```
rx_err_t err = spi_transfer(data, len);
switch (err) {
    case k_rx_ok:
        // Success path
        break;
    case k_rx_err_timeout:
        // Retry logic
        retry_transfer();
        break;
    case k_rx_err_nack:
        // Device not responding
        reset_device();
        break;
    default:
        // Unexpected error
        log_error(err);
        break;
}
```

Macro-Based Error Propagation:

```
#define RX_RETURN_ON_ERROR(err, tag, msg) \
    do { \
        rx_err_t _err = (err); \
        if (_err != k_rx_ok) { \
            rx_log_error((tag), (msg)); \
            return _err; \
        } \
    } while (0)

rx_err_t init_system(void) {
    RX_RETURN_ON_ERROR(init_gpio(), "INIT", "GPIO init failed");
    RX_RETURN_ON_ERROR(init_spi(), "INIT", "SPI init failed");
    return k_rx_ok;
}
```

Note

This type is used by ALL functions that can fail. Consistent usage across the codebase enables systematic error handling and logging.

Warning

Never cast arbitrary integers to rx_err_t. Only use predefined k_rx_∗ constants or values returned from error conversion functions.

Attention

Functions returning rx_err_t MUST document all possible return values using

Return values

tags	in their Doxygen documentation.

Memory Layout:

- Size: 4 bytes
- Alignment: 4 bytes (native word on ARM Cortex-R4)
- Signedness: Signed (two's complement)
- Range: -2,147,483,648 to 2,147,483,647 (only 0-0x5FF currently used)

Performance:

- Return value overhead: 0 cycles (returned in register r0)
- Comparison overhead: 1 cycle (CMP r0, #0)
- No heap allocation required

See also

rx_err_codes_t Enumeration of all defined error codes

rx_err_is_ok() Helper to check for success

rx_err_is_error() Helper to check for failure

rx_err_from_threadx() Convert ThreadX codes to rx_err_t

Since

Version 1.0.0

Definition at line 240 of file rx_err.h.

7.5.3 Enumeration Type Documentation

7.5.3.1 rx_err_codes_t

enum rx_err_codes_t : uint16_t

Comprehensive error code enumeration for RX72N firmware.

Type-safe error codes using C23 typed enum syntax with explicit uint16_t underlying type. All error codes are compile-time constants organized into logical categories for systematic error handling and debugging.

7.5.3.2 Error Code Categories

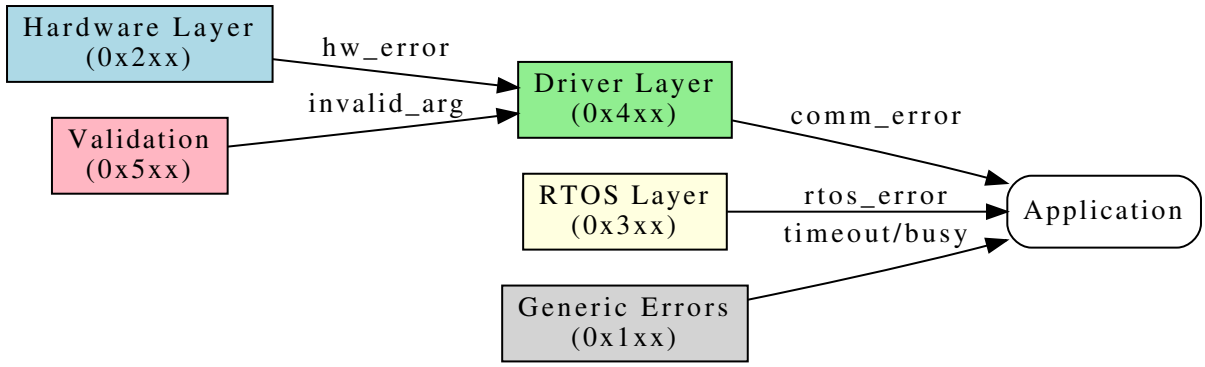
The error code space is partitioned into non-overlapping ranges:

Range	Category	Count	Purpose
0x000	Success	1	Operation completed successfully

Range	Category	Count	Purpose
0x100-0x1FF	Generic	15	General-purpose errors (null, timeout, etc.)
0x200-0x2FF	Hardware	9	Peripheral, GPIO, and sensor errors
0x300-0x3FF	RTOS	6	ThreadX and synchronization errors
0x400-0x4FF	Communication	8	SPI, UART, I2C, and protocol errors
0x500-0x5FF	Validation	4	Input validation and checksum errors

7.5.3.3 Error Category Flow

Typical error propagation through the system:



7.5.3.4 Error Handling Patterns

Pattern 1: Direct Comparison

```
rx_err_t err = motor_start();
if (err == k_rx_ok) {
    // Success
} else if (err == k_rx_err_estop) {
    // Emergency stop active - cannot start
} else {
    // Other error - log and handle
}
```

Pattern 2: Category Check

```
rx_err_t err = spi_transfer();
if ((err & 0xF00) == 0x400) {
    // Communication error category
    restart_communication();
} else if ((err & 0xF00) == 0x200) {
    // Hardware error category
    reset_hardware();
}
```

Pattern 3: Error Logging with Context

```
rx_err_t err = init_subsystem();
if (err != k_rx_ok) {
    rx_log_error("INIT", "Subsystem init failed: 0x%03X", err);
    // Hex dump shows category: 0x2xx = hardware, 0x4xx = comm, etc.
}
```